MULTI-MODAL INTELLIGENT TRAFFIC SIGNAL SYSTEM (MMITSS)

Field Applications-User Manual

THE UNIVERSITY OF ARIZONA

Version 2.0
JULY 20, 2017

Contents

1. Document Overview 3
2. Overview of MMITSS 4
   2.1. MMITSS Architecture 4
   2.2. Design and Implementation of the MMITSS Prototype 6
   2.3. Overview of the MMITSS Prototype 7
   2.4. Architecture of MMITSS 8
      2.4.1. Architecture Components 9
3. MMITSS Deployment on Road-Side Equipment (RSE) 15
   3.1. Pre-Requisites 15
   3.2. Installation of Road Side Equipment 15
   3.3. Hardware Connections 17
   3.4. Running MMITSS Applications On RSE 18
   3.5. Component Specific MMITSS Applications 24
      3.5.1. General MMITSS Applications (Applicable to All Components) 24
      3.5.2. Intelligent Signal Control (I-SIG) 31
      3.5.3. Traffic Signal Priority (TSP) 32
      3.5.4. Real-Time Performance Observer (PERF-OBS) 36
      3.5.5. Mobile Accessible Pedestrian Signal System (PED-SIG) 39
4. MMITSS Deployment on On-Board Equipment (OBE) 40
   4.1. Pre-Requisites 40
   4.2. Running MMITSS applications On OBE 40
   4.3. Component Specific MMITSS Applications 47
      4.3.1. General MMITSS Applications 47
      4.3.2. Traffic Signal Priority (TSP) 47
5. Troubleshooting 50
   5.1. Reduced Response-Time of System 50
   5.2. Missing MMITSS Components 51
6. References 53
7. Appendices 55
   7.1. Construction and an example of map file 55
      7.1.1. General Description of the Process for Development of the MAP Data 55
      7.1.2. Conventions Used In MAP Deployment 55

7.1.3. MAP Description File of Daisy Mountain Dr and Gavilan Peak 57
8. Record of Changes 68

List of Figures

Figure 1. MMITSS Physical Architecture. 5
Figure 2: MMITSS Architecture 8
Figure 3: MMITSS User Interface 11
Figure 4: Pole Mounted RSE 16
Figure 5: PoE Connections 17
Figure 6: PoE Connections - Pictures 17
Figure 7: WinSCP Icon 18
Figure 8: Login Window of WinSCP 18
Figure 9: Credentials Confirmation for Saved Accounts (WinSCP) 19
Figure 10: WinSCP User Interface and Location of PuTTY 20
Figure 11: PuTTY - Accessing Root Directory 21
Figure 12: PuTTY - Accessing Any Known Directory 21
Figure 13: PuTTY - Starting MMITSS Script 22
Figure 14: PuTTY - Stopping MMITSS Script 22
Figure 15: Example of MMITSS Script 23
Figure 16: Lane Phase Mapping 30
Figure 17: Sample Network for Lane Movement Mapping 38
Figure 18: Communication Architecture for MMITSS Pedestrian Application 39
Figure 19: OBE - Connections 40
Figure 20: WinSCP Icon 40
Figure 21: WinSCP Login Window 41
Figure 22: WinSCP Credentials Confirmation for Saved Accounts 41
Figure 23: Entering PuTTY 42
Figure 24: WinSCP User Interface 43
Figure 25: PuTTY - Accessing Root Directory 44
Figure 26: PuTTY - Accessing Any Known Directory 44
Figure 27: PuTTY - Starting MMITSS Script 45
Figure 28: PuTTY - Stopping MMITSS Script 45
Figure 29: Example of MMITSS Script 46
Figure 30: WinSCP - Selecting All Log Files 50
Figure 31: WinSCP - Deleting All Log Files 51
Figure 32: PuTTY - Checking All Running Applications 52
Figure 33: Area Map - Daisy Mountain Dr and Gavilanb Peak 57
Figure 34: Intersection Map - Daisy Mountain Dr and Gavilan Peak 58

1. DOCUMENT OVERVIEW

This document is an installation and field user guide for the *Multi-Modal Intelligent Traffic Signal System* (MMITSS) applications. This document contains the detailed configuration usage instructions for each of the desired MMITSS components.

This user guide begins with the general introduction to MMITSS in the Section 2. Instructions for deployment of MMITSS on the RSE are provided in Section 3, followed by instructions for MMITSS deployment on the OBE, in Section 4. A MMITSS troubleshooting guide for commonly faced problems in deployment is provided in Section 5. A guide to create a Map file for MMITSS is provided as an Appendix to this user manual.

2. OVERVIEW OF MMITSS

The Multi-Modal Intelligent Traffic Signal System (MMITSS) project is part of the Connected Vehicle Pooled Fund Study (CV PFS) entitled "Program to Support the Development and Deployment of Connected Vehicle System Applications." The CV PFS was developed by a group of state and local transportation agencies and the Federal Highway Administration (FHWA). The Virginia Department of Transportation (VDOT) serves as the lead agency and is assisted by the University of Virginia's Center for Transportation Studies, which serves as the technical and administrative lead for the PFS.

The USDOT has identified six mobility application bundles under the Dynamic Mobility Applications (DMA) program for the connected vehicle environment where high-fidelity data from vehicles, infrastructure, pedestrians, etc. can be shared through wireless communications[1]. Each bundle contains a set of related applications that are focused on similar outcomes. Since a major focus of the CV PFS members – who are the actual owners and operators of transportation infrastructure – lies in traffic signal related applications, the CV PFS team is leading the project entitled "Multi-Modal Intelligent Traffic Signal System" in cooperation with US DOT's Dynamic Mobility Applications Program. As one of the six DMA application bundles, MMITSS includes five applications: Intelligent Traffic Signal Control (I-SIG), Transit Signal Priority (TSP), Mobile Accessible Pedestrian Signal System (PED-SIG), Freight Signal Priority (FSP), and Emergency Vehicle Preemption (PREEMPT).

The MMITSS prototype was developed based on traffic controllers using the NTCIP communications protocol and new algorithms for providing priority control (emergency vehicle, transit, and truck priority) and intelligent signal control (e.g. adaptive control using connected vehicle data).

2.1. MMITSS ARCHITECTURE

The Physical MMITSS architecture is shown in Figure 1 as a UML Deployment Diagram. In the Unified Modeling Language (UML), nodes are shown as 3D blocks and represent physical devices that have a processor (at least one), memory, and physical interfaces (e.g. Ethernet, RS-232, or wireless – 3G/4G, 5.9 GHz DSRC, or other such as CAN-bus). The basic architecture is applicable at each MMITSS controlled intersection. In a system that consists of several intersections, each intersection would have an RSE Radio and traffic control equipment as shown in the deployment diagram. There would generally be only one instance of the Traffic Management System, but there could be multiple instances of the Fleet Management System (e.g. Transit Fleet, commercial truck fleet, emergency vehicles).

[1] http://www.its.dot.gov/research_archives/dma/dma_plan.htm

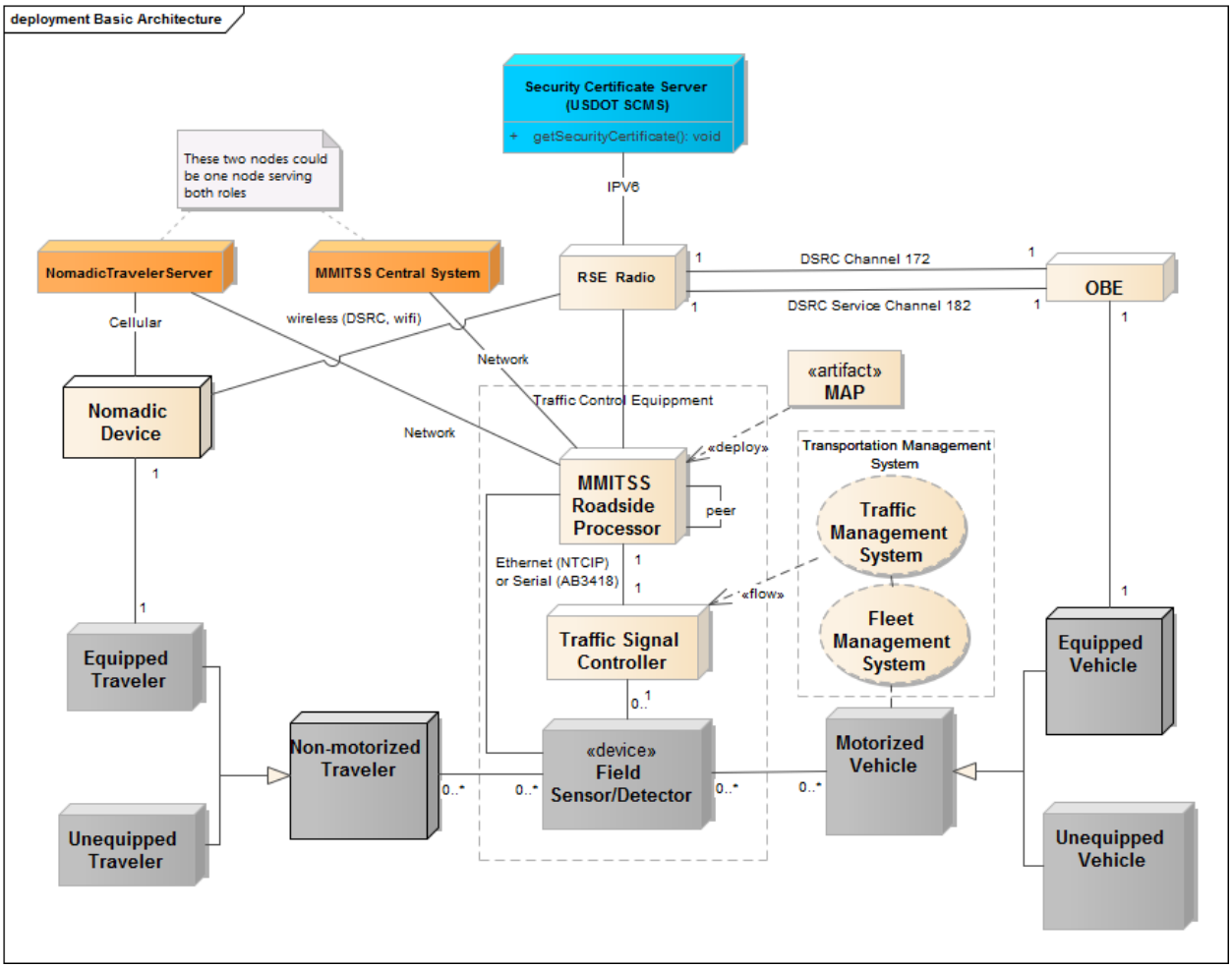


FIGURE 1. MMITSS PHYSICAL ARCHITECTURE.

The nodes have been shaded such that the light-colored nodes are part of the connected vehicle system. The Traffic Management and Fleet Management systems are shown as UML collaborations and aren't part of the MMITSS system. The gray colored nodes represent the vehicles, travelers and infrastructure sensors (e.g. inductive loops or video detectors). The Security Certificate Server (e.g. USDOT Security Credential Management System – SCMS) is interfaced to the RSE using a backhaul network and is used to provide security certificates to trusted OBEs and a user revocation list to the RSE. The orange colored nodes are the MMITSS Central System and Nomadic Traveler Server as described below. These two nodes may be realized as a single node for the test bed implementations.

In this view of the system, there are two types of travelers – motorized vehicles and non-motorized travelers. Motorized vehicles consist of passenger vehicles, trucks, transit vehicles, emergency vehicles, and motorcycles. This type of traveler includes any vehicle that must be licensed to operate on the public roadway. Non-motorized travelers include pedestrians, bicyclists, and other modes such as equestrians that are not required to be licensed to operate on the public roadway. These travelers are either unequipped or equipped, meaning that they have some type of OBE (On-Board Equipment) or nomadic device that is connected vehicle (or MMITSS) aware and can operate as part of the traffic control system.

Motorized vehicles can be part of a fleet management system such as a transit management system, commercial freight management system, emergency vehicle dispatch system, or taxi dispatch, which is shown as a UML collaboration (oval in Figure 1) meaning that a collection of entities work together to perform the traffic management functions, but there may be many different systems involved in this collaboration.

The infrastructure based traffic signal control equipment consists of the traffic signal controller, field sensors/detectors, an optional MMITSS Roadside Processor (MRP) and the Roadside Equipment (RSE). The RSE Radio is the hardware device that is responsible for managing all of the 5.9 GHz DSRC communications between the vehicles and the infrastructure. Two channels are utilized for communications between the RSE and the vehicle on-board equipment (OBE). Channel 172 is called the safety-of-life channel and is used by vehicle for broadcasting basic safety messages (BSM) and by the infrastructure (RSE) for broadcasting the map message (MAP) and the signal phase and timing message (SPaT). Channel 182 is used for exchanging the signal request messages (SRM) and signal status messages (SSM). The RSE hardware used for the MMITSS development and testing was supplied by Savari, Inc. and is called the StreetWave unit.

The OBE is a hardware device deployed on the vehicle. MMITSS was developed and tested using Savari MobilWave units (called aftermarket safety devices - ASD) for the OBE. These units are general purpose and provide a powerful and flexible platform for development and testing. The OBE devices exchange messages with the RSE based on the SAE J2735 (2009) standard using the Dedicated Short Range Communications standards – IEEE 1609.

The MMITSS system was tested using Econolite ASC/3 (or Cobalt) traffic signal controllers. The Econolite ASC/3 and the Cobalt controllers are based on NEMA standards and support NTCIP over Ethernet communications. MMITSS has been tested with McCain NTCIP controllers at the FHWA Saxton Transportation Lab. The MMITSS Roadside Processor (MRP) is a Linux based general-purpose computer and is currently based on the Econolite Connected Vehicle Co-Processor module.

The MMITSS Roadside Processor (MRP) is a general-purpose computer that can host the core intersection level infrastructure applications for MMITSS. The RSE contains (deploys) the MAP artifact, which is the digital description of the intersection geometry and associated traffic control definitions.

Both motorized and non-motorized travelers can be detected by the Field Sensor/Detector node at the intersections using a variety of detection technologies, including inductive loop detectors, video detection, microwave, radar, pedestrian push button, etc. The detection system at an intersection provides information to the traffic signal controller that stimulates the control algorithms. For example, a vehicle that triggers a detector will call a signal control phase for service or extension. A pedestrian may activate a pedestrian push button to request the traffic signal pedestrian interval associated with a crosswalk movement. The direct communications path from the field sensor to the MRP allows the communication of information from the sensor. This information might include vehicle count from presence detectors or possibly vehicle trajectories from radar based sensors in the future. [Implementation Note: No direct sensor data was used in the MMITSS implementation.]

2.2. DESIGN AND IMPLEMENTATION OF THE MMITSS PROTOTYPE

The initial implementation of MMITSS utilized the processing capabilities of the Savari RSE and didn't utilize a separate roadside processor (MRP). Since the software was designed as a collection of software

components that share data as needed, e.g. a peer-to-peer data architecture, the different software components can be built and deployed on any set of nodes (e.g. a processor with memory) including the Savari RSE or a separate MRP. The details of the MMITSS prototype can be found in the Detailed Design Report (http://www.cts.virginia.edu/wp-content/uploads/2014/05/Task4._SystemDesign_3_Revised_v.2.pdf) and in the US DOT Open Source Application Development Portal (http://www.itsforge.net/).

2.3. OVERVIEW OF THE MMITSS PROTOTYPE

The MMITSS prototype was developed with the philosophy that connected vehicle data, both vehicle basic safety messages and priority request messages, would provide significant additional information that could be used to develop a new generation of traffic signal control algorithms. Goodall (2013) demonstrated that connected vehicle data could improve traffic signal control performance. Together, with recent advances in adaptive traffic signal control, such as OPAC (Gartner, 1983), RHODES (Mirchandani and Head, 2001), and ACS Lite (Gettman, et. al., 2006) and the results of the NCHRP 3-66 project (Urbanik, et. al, 2003), there has been significant advancement in the development of traffic control algorithms for both general vehicles (adaptive signal control) and for special classes of vehicle (priority control). In addition, it was recognized that the market penetration rate of connected vehicles would be low for several years, but priority eligible vehicle fleets could be good candidates for early adoption projects – such as transit priority corridors, freight priority corridors, and emergency vehicle systems. If MMITSS could provide effective priority control with low market penetration rates for personal vehicles, then the entire traffic management system could benefit as the market penetration increases.

One aspect of this strategy was the recognition that actuated traffic signal control would continue to be the standard at most traffic signals, so MMITSS should be able to integrate innovative priority control with actuated signal control. Coordination is also a critical traffic control strategy and MMITSS should be able to integrate priority control with coordination and actuation. As such, coordination was considered to be a form of priority which could be addressed in a unified priority control framework.

An important concept adopted in the MMITSS prototype was the concept of multiple levels of importance for different modes in traffic control. An operating agency should have the ability to determine which mode of priority should be the most important and which should be the least important when serving multiple requests at one time. For example, in one corridor, coordination might be the most important consideration during the AM and PM peak commute times or during an incident on a parallel freeway. In the off-peak times, the priority might favor transit vehicles in an urban shopping/school area or might favor freight in a commercial warehouse/factory area. The ability to establish a “priority policy” would allow the operating agency to have a powerful tool for traffic management.

Finally, the recent development and agency adoption of the NTCIP communication standards has provided a favorable path for deployment of MMITSS. NTCIP compliant traffic signal controllers can easily provide the current signal timing plan data to MMITSS and can interface with the MMITSS algorithms. The standards provide a well-defined and common interface that is now widely used across the United States. Most major traffic signal controller manufacturers support NTCIP and NTCIP provides the interfaces necessary for working with the controllers.

In the following sections, an overview of the MMITSS system is presented. The goal in this section is to familiarize the reader with the design and algorithm concepts used in the MMITSS prototype. Details of

the software implementation can be found in the MMITSS Detailed Design document. The following section is divided into two main subsections. The first presents the MMITSS architecture and describes the software components that provide the platform for the MMITSS traffic control algorithms. The second section provides an introduction to the adaptive signal control, the priority control, and the performance observer components. The MMITSS Central System was not fully developed and tested in the Phase II prototype effort, but represent key architecture and approach components that would be required in any deployment.

2.4. ARCHITECTURE OF MMITSS

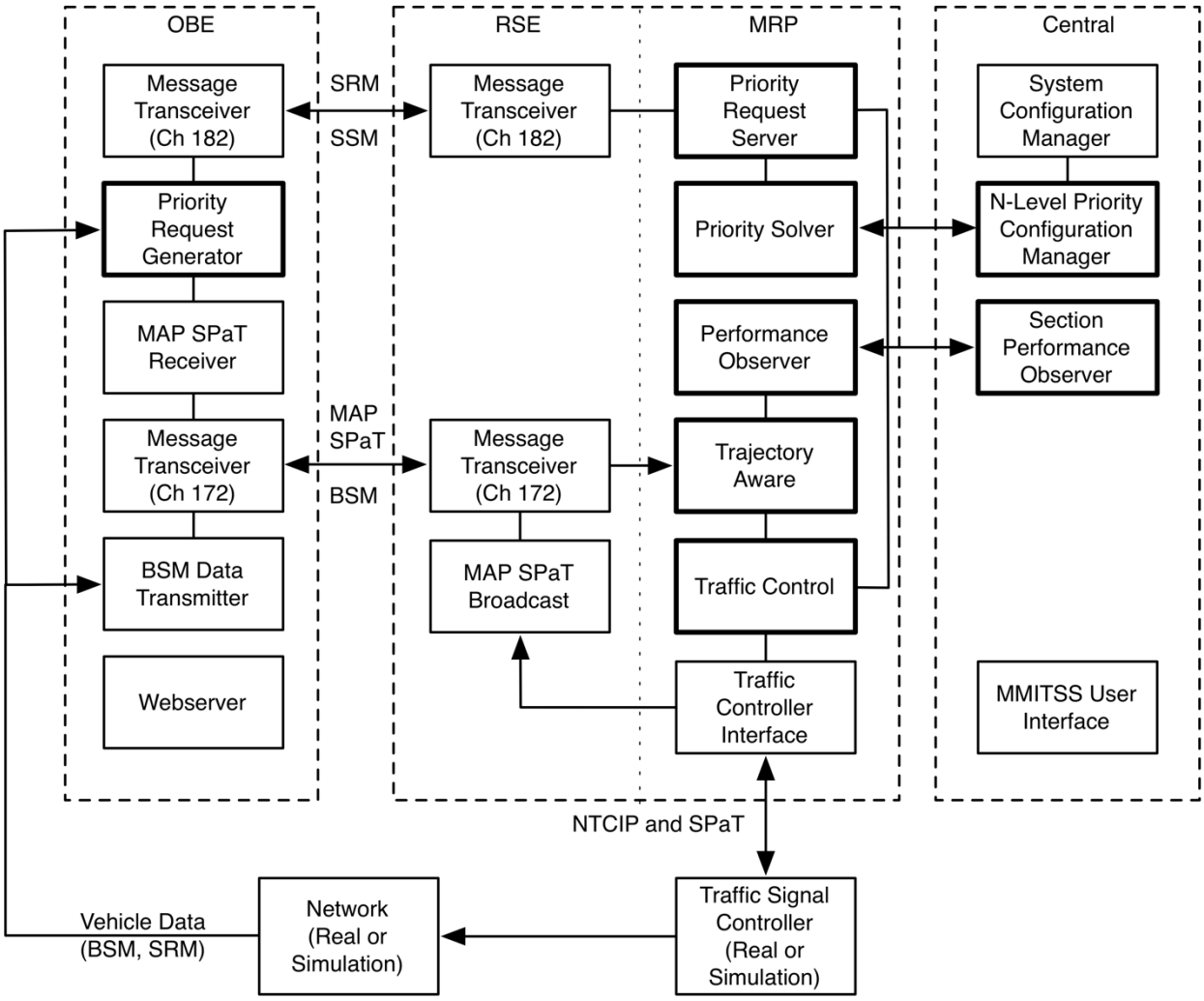


FIGURE 2: MMITSS ARCHITECTURE

The MMITSS architecture is shown in Figure 2. The architecture was designed based on a component based, or distributed, software design where the components use peer-to-peer communications (sockets) to share information. Figure 2 shows three key MMITSS architecture elements: the on-board Equipment (OBE), roadside processor and equipment (RSE and MRP), and the Central system. Software components are deployed on each of the key architecture elements. There are two kinds of components: MMTISS traffic control algorithms (shown with think black borders) and MMTISS interface and general connected vehicle components. The interface components allow the MMITSS system to acquire data and actuate

controls, as well as to interface to the two DSRC communication channels that are used. The MMITSS traffic control components implement the new control algorithms developed to use the connected vehicle data.

2.4.1. Architecture Components

The architecture includes six software components that run on the OBE, ten software components that run on the RSE, or on the MMITSS Roadside Processor (MRP), and four that run on the central system (a Windows based PC). In the current implementation, all RSE and MRP components are run on the RSE (Savari Streetwave). The peer-to-peer socket interfaces allow the components to be installed on any processor in the subnetwork and configured to communicate with the desired peer component(s). For example, all the algorithm components (thick black lines in Figure 2**Error! Reference source not found.**) could be run on a separate processor, or some could be on the RSE and some on the MRP, or all on the RSE – as was tested in the Arizona Field Test.

Applications, including adaptive signal control and dilemma zone protection, utilize only BSMs while coordination and signal priority utilize SPaT, MAP, SRM and SSM. Two DSRC channels 172 and 182 are used to send/receive different types of messages. Channel 172 is the safety channel, which is used to transmit safety related messages (e.g., BSM, SPaT and MAP). Channel 182 is one of the available service channels and was selected to transmit priority application specific messages (e.g., SRM and SSM for priority control). Any of the other service channels could be used if the transmitting and receiving radios are configured for this use.

This architecture has been implemented in both simulation (VISSIM 6.0 with the Econolite ASC/3 software-in-the-loop (SIL) signal controller) and in the field using NTCIP compliant signal controllers.

2.4.1.1. MESSAGE TRANSCEIVER (OBE AND RSE)

The Message Transceiver application is configurable to utilize either of the two 5.9GHz radios. One is configured to use channel 172 and the other to use channel 182. Channel 172 is used by the OBE to broadcast basic safety messages (BSM) and to receive MAP and SPaT messages that are broadcast by the RSE (MAP SPaT Broadcast component). Channel 182 is used to broadcast and receive Signal Request Messages (SRM) and Signal Status Messages (SSM). The Message Transceiver interfaces to the native IEEE 1609 WAVE stack on the Savari RSE, but could also be implemented on other roadside units.

2.4.1.2. MAP SPAT RECEIVER (OBE)

The MAP SPaT Receiver is deployed on the OBE and is responsible for receiving and unpacking the J2735 (2009) MAP and SPaT messages and making the data available to the Priority Request Generator. The MAP SPaT receiver can supply MAP and SPaT data to other components on the OBE if desired. The MAP data is stored in a file with labeled with the name of the intersection for instances where there may be more than one active MAP received.

2.4.1.3. BSM DATA TRANSMITTER (OBE)

The BSM Data Transmitter is responsible for building the J2735 (2009) Basic Safety Message (BSM). For the MMITSS prototype, the BSM contains a temporary identification number (ID, the vehicle's GPS position, speed (from GPS) and the vehicle type, which is read from a file and can be changed for different vehicle types. The message is built 10 times per second and sent to the Message Transceiver for broadcast on Channel 172. Currently, the BSM Data Transmitter does not read vehicle data from the vehicle's CAN

bus and it does not do any position correction. The position accuracy was sufficient for field testing in the Arizona Connected Vehicle Test Bed where lane level positions were generally received.

2.4.1.4. TRAFFIC CONTROLLER INTERFACE (RSE, MRP)

The Traffic Control Interface component is responsible for all communications with the traffic signal controller. There are three main communication interactions between the traffic signal controller and the Traffic Controller Interface component: 1) register and receive SPaT data, 2) query the controller for configuration data, and 3) command the controller to implement the desired traffic control schedule. All communications with the traffic controller (Econolite ASC/3 or Cobalt) are through the local Ethernet network. The Econolite ASC/3 controller is configured with the IP address and port where it can stream SPaT messages. The SPaT messages are defined according to the Battelle (2012) Signal Phase and Timing specification that was developed for FHWA. SPaT data is received, unpacked and made available to the Traffic Control and Priority Request Server components 10 times per second. [Note: The Priority Solver (RSE, MRP) currently queries the traffic controller for status and configuration data. This communication will be relocated to the Traffic Control Interface in the near future].

The controller configuration data is queried when it is started and provides the Phase Timing Interval Data defined in. [Note: the frequency of this query can be changed to allow MMITSS to be aware of changes to the controller configuration by a traffic technician using the controller front panel or by a central traffic management system.] In addition, Detector Parameter configuration is read on start up, and Detector Data (Phase Call, Occupancy, and Volume) are read once per second. This data is made available to Traffic Control, Priority Request Server, and the Performance Observer components as input for their functions. All controller configuration and detector data is queried using NTCIP 1202 objects (2005).

The commands from AZ MMITSS to the traffic signal controller are implemented using NTCIP Phase Control Objects (Table 1). AZ MMITSS utilizes the controller configuration together with the data from vehicle detectors, connected vehicle basic safety messages and signal request messages to determine an optimal signal timing schedule that consists of the phase sequence and durations. The Traffic Control Interface component monitors the controller status and sends the appropriate NTCIP Phase Control command to the controller to implement the signal timing schedule. The optimal signal timing schedule is updated by the MMITSS Traffic Control and Priority Request Server components as new information is available. As the schedule is updated the Traffic Control Interface component issues the commands to the traffic signal controller.

TABLE 1

Data Object	Parameters	NTCIP 1202 Protocol Objects	Usage
PhaseTimingIntervals	PhaseNumber PhaseWalk PhasePedestrianClear PhaseMinimumGreen PhasePassage PhaseMaximum PhaseYellowChange PhaseRedClear	Phase Parameters	Input to *MRP_TrafficControl* & *MRP_PRS_PriorityServer* for phase allocation

DetectorParameters	DetectorID DetectorType DetectorCallPhase NTCIPOccupancy NTCIPVolume	Detector Parameters	Input to *MRP_PerformanceObser* for performance estimation *MRP_TrafficControl* for phase allocation
PhaseControl	PhaseOmit PhaseHold PhaseForceOff VehicleCall PedCall	Phase Control	Output from MRP_TrafficControlInterface to the traffic signal controller

2.4.1.5. WEBSERVER

A webserver is used on the OBE to provide a simple and flexible visualization of the status of MMITSS during field testing. Figure 3 shows an example of the MMITSS visualization. The webpage shows the current signal status by phase and pedestrian interval as well as the status of the MAP and the Priority Request that has been broadcast. It also shows two priority requests that were received by the Priority Request Server and acknowledged through the [revised] Signal Status Message. The emergency vehicle request is overriding the transit vehicle request – as indicated by the red 'x' in the Active column of the table. The in-lane and out-lane, requested phase(s), estimated arrival time, time of desired service, estimated departure time, and vehicle status is shown. All of this data is available through the [revised] J2735 Signal Status Message (SSM).

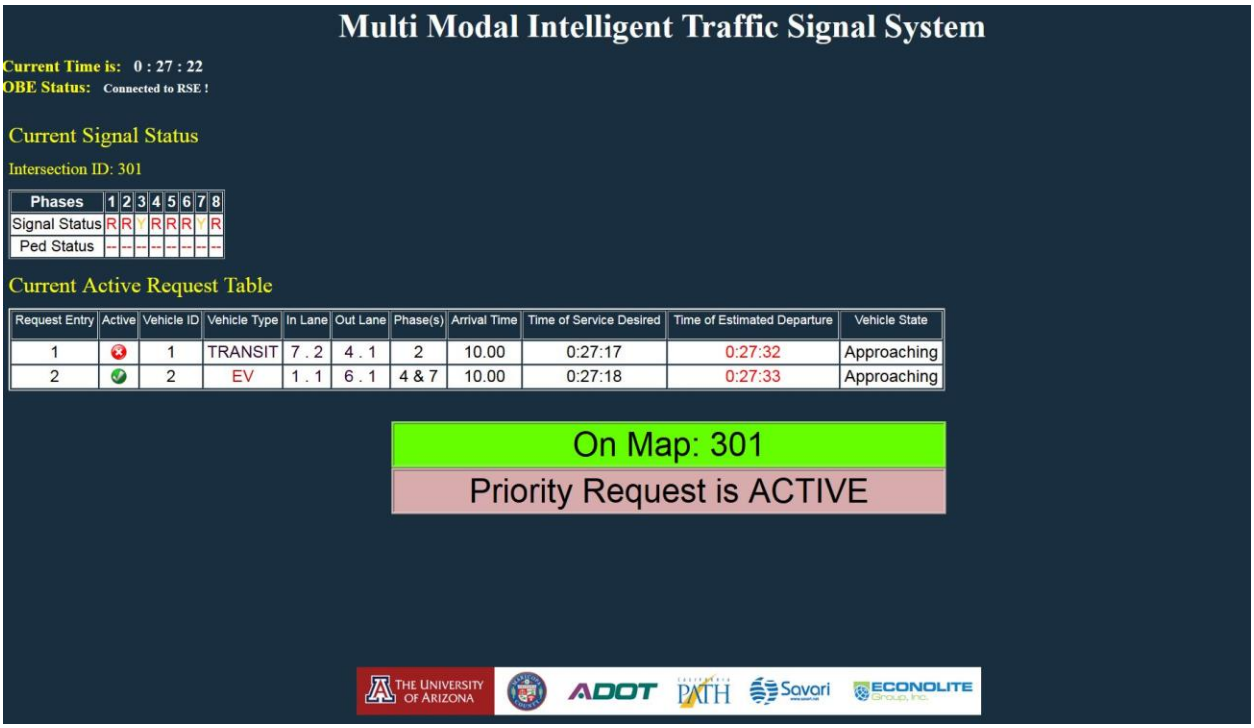


FIGURE 3: MMITSS USER INTERFACE

2.4.1.6. PRIORITY REQUEST GENERATOR (OBE)

The Priority Request Generator (PRG) is deployed on the OBE. The PRG embodies the vehicle based logic that determines when to send a priority request and the contents of the request message. In the AZ MMITSS prototype it is assumed that any priority eligible vehicle, e.g. emergency vehicles, transit vehicles, and trucks, can send a request when they are approaching an equipped intersection. This assumption would require additional development in a deployment. For example, an emergency vehicle would only request EV priority when the lights and sirens are in use. A transit vehicle might only send a request when it was behind schedule with significant occupancy.

The Priority Request Generator (PRG) checks to see if the MAP SPaT Receiver has new map data. Once a MAP, or several MAPs, are received the PRG determines which MAP describes the intersection that the vehicle is approaching and which lane/approach the vehicle is located. Once this is determined, the PRG will compute the travel time to the stop bar (from the MAP data and the current position) and will build and send the Signal Request Message. The PRG will wait for a Signal Status Message (SSM) to confirm receipt of the request. If no SSM is received, or an SSM is received and the vehicle's request isn't in the table of active requests, the PRG will update the SRM and send it again. If the vehicle speed changes significantly, the estimated time of arrival is updated and an updated request is sent. Once the vehicle crosses the stop bar, a cancel request is sent. Once a SSM is received showing the canceled request is no longer active, the PRG will check any available MAPs or wait for a new MAP to be received.

2.4.1.7. TRAJECTORY AWARE (RSE, MRP)

The Equipped Vehicle Trajectory Aware component is responsible for collecting data from each of the connected vehicles that is broadcasting Basic Safety Messages (BSM) when in the range of an RSE. Each BSM received by the Message Transceiver (Channel 172) is forwarded to the Trajectory Aware component. The Trajectory Aware component checks to see if the vehicle is within the geo-fence area created by the MAP GPS waypoints and creates a record for each vehicle's temporary ID that is received and within the geo-fence. Checking the geo-fence location of each vehicle allow the system to filter out vehicles that are in nearby parking lots or other non-traffic controller facilities.

Each record is time stamped and contains the position, speed, vehicle type information, traffic control phases, and estimated time of arrival (ETA) at the stop bar (assuming current speed). If a vehicle changes its temporary ID, a new record will be created for the new vehicle ID. Vehicle data is retained for 5 minutes after the vehicle leaves the RSE range or after it changes its temporary ID. This data is deleted as part of the privacy assurance requirements in the design. The collection of all vehicles that are sending BSM is available to any other AZ MMITSS component that requests data about the location, and trajectory history, if each connected vehicles. Trajectory Aware provides data for Traffic Control and the Performance Observer.

2.4.1.8. TRAFFIC CONTROL (RSE, MRP)

The Traffic Control component is responsible for the allocation of green time to the traffic control phases based on the information available in Trajectory Aware and from vehicle detector data from the Traffic Control Interface component. Traffic Control was developed to provide traffic adaptive control using concepts from the COP algorithm (Sen and Head, 1997) that was developed as part of the RHODES (Mirchandani and Head, 2001) and to provide dilemma zone protection. The AZ MMITSS Traffic Control component is based on three algorithms: 1) Estimation of the location of unequipped vehicles (called EVLS), 2) a phase allocation algorithm that determines the duration of the green phases based on the data

from trajectory aware and the estimated locations of unequipped vehicles, and 3) a dilemma zone protection algorithm. Details of these algorithms can be found Feng, et. al. (2016).

2.4.1.9. PRIORITY REQUEST SERVER (RSE, MRP)

The Priority Request Server component and the Priority Solver component provide the multi-modal priority control for priority requests from multiple vehicles. The AZ MMITSS approach to priority utilizes a new approach to priority and preemption rather than implementing legacy algorithms that exist in current traffic signal controllers. The new approach is based on research from the NCHRP 3-66 project "Traffic signal state transition logic using enhanced sensor information" (Urbanik, et. al., 2003) that investigated improved approaches to priority control. The algorithms implemented in the AZ MMITSS system use information in the connected vehicle Signal Requests Messages that are received from multiple vehicles of multiple modes. At any time, there may be several active requests that are simultaneously considered through a mathematical programming model that captures the ring-barrier and phase-interval based structure of the traffic signal controller. The formulation considers how to best serve all active requests based on a policy (N-level policy) set by the operating agency by selecting the mode priority weights in the mathematical programming model. The model is formed and managed by the Priority Request Server and solved by the Priority Solver. The Priority Solver uses an open source optimization solver – the GNU Linear Programming Kit (GLPK) to solve the optimization each time a new request is received, updated, or cancelled. The output of the Priority Request Server is a flexible signal timing schedule that the Traffic Control component can take as input when allocating phase green time based on the estimated vehicle arrivals. Details of the algorithms can be found in Zamanipour (2018).

2.4.1.10. PERFORMANCE OBSERVER COMPONENT

The Performance Observer component is responsible for acquiring data from other components, including the Trajectory Aware and the Traffic Controller Interface components, to compute observed performance measures. The Performance Observer includes estimation, managing, and archiving the data. All the acquired performance observations and metrics are collected and reported in terms of peak period, all day, or specially defined time period.

As stated in the design document the Basic Safety Message contains the vehicle temporary ID, latitude, longitude, elevation, speed, heading, and vehicle size. It doesn't have the vehicle type information, although it can be determined according to the Signal Request Message. The high-resolution data (i.e. received BSMs every 0.1 second) are used to reconstruct the vehicle trajectories.

TABLE 2 PERFORMANCE MEASURES

Performance Metric	Abbreviation	Unit	Data Source
Travel Time	TT	Second	Trajectory Aware
Delay	D	Second	Trajectory Aware
Travel Time Variability	TTV	Second	Trajectory Aware
Delay Variability	DV	Second	Trajectory Aware
Queue Length	QL	Meter/number of vehicles	Trajectory Aware
Number of Stops	NS		Trajectory Aware
Volume	V		Traffic Controller Interface

Occupancy	O	%	Traffic Controller Interface
DSRC Range	RNG	Meter	Trajectory Aware
Packet Loss Rate	PLR	%	Trajectory Aware
Market Penetration Rate	MPR	%	Trajectory Aware Trajectory Aware Traffic Controller Interface

Table 2 summarizes the performance measures being calculated and estimated in this component with their associated sources of data. The details of the algorithms used to compute the performance measures can be found in Khoshmagham (2016). Table 3 summarizes 16 MMITSS applications with their hardware location (e.g. Road-Side Equipment) and the MMITSS functions that it supports.

TABLE 3: MMITSS APPLICATIONS AND THEIR LOCATIONS

Sr No	Application Name	Location	MMITSS Components
1	MMITSS_MRP_EquippedVehicleTrajectoryAware	RSE	I-SIG, PERF-OBS
2	MMITSS_MRP_MAP_SPAT_Broadcast	RSE	TSP
3	MMITSS_MRP_Priority_Solver	RSE	TSP
4	MMITSS_MRP_PriorityRequestServer	RSE	TSP
5	MMITSS_MRP_Signal_Control	RSE	I-SIG
6	MMITSS_MRP_TrafficControllerInterface	RSE	I-SIG, TSP
7	MMITSS_PED_MAP_Broadcast	RSE	PED-SIG
8	MMITSS_MRP_PerformanceObserver	RSE	PERF-OBS
9	MMITSS_MRP_Priority_Webserver	RSE	PERF-OBS, TSP
10	MMITSS_MRP_PedRequestServer	RSE	PED-SIG
11	MMITSS_RSE_Message_Transceiver	RSE	I-SIG, TSP, PERF-OBS
12	MMITSS_OBE_BSMData_Transmitter	OBE	I-SIG, PERF-OBS
13	MMITSS_OBE_MAP_SPAT_Receiver	OBE	TSP
14	MMITSS_OBE_PriorityRequestGenerator	OBE	TSP
15	MMITSS_OBE_Webserver	OBE	TSP
16	MMITSS_OBE_Message_Transceiver	OBE	I-SIG, TSP, PERF-OBS

3. MMITSS DEPLOYMENT ON ROAD-SIDE EQUIPMENT (RSE)

3.1. PRE-REQUISITES

Prior to implementation of MMITSS on any intersection, availability of following must be met:

1. Savari StreetWAVE Road Side Equipment, produced by Savari Inc.
2. The MMITSS application package installed on RSU.[2]
3. Access credentials (user name and password) of Savari StreetWAVE RSE.
4. Econolite ASC/3 Signal Controller.[3]
5. MMITSS Powered-Over-Ethernet Kit.
6. External computer, installed with WinSCP to interface with RSE.

3.2. INSTALLATION OF ROAD SIDE EQUIPMENT

The installation of the RSE is dependent upon local design, policies and available infrastructure. It can be mounted directly on a traffic pole or mast arm to ensure radio communication objectives can be met. Steps must be taken to limit the RSE signal within the communication zone to the maximum extent practicable. It may employ an antenna with height exceeding 8 meters but not exceeding 15 meters from the roadway bed surface. RSE installation must also meet the restrictions imposed by Electronic Code of Federal Regulations (Title 47-Chapter I-Subchapter D-Part 90-Subpart M- §90.377). A link to these regulations is provided below. Figure 4 shows an example of the pole mounted RSE.

http://www.fdot.gov/traffic/Doc_Library/PDF/USDOT%20RSU%20Specification%204%201_Final_R1.pdf

It is recommended that MMITSS applications should be installed and tested in the shop before mounting the RSU in the field.

[2] MMITSS Application Package can be obtained from Systems and Industrial Engineering Department of The University of Arizona. Contact Dr. Larry Head at klhead@email.arizona.edu

[3] The controller should be NTCIP compliant. Currently Econolite ASC/3, Econolite Cobalt, and the McCain controllers have been tested.

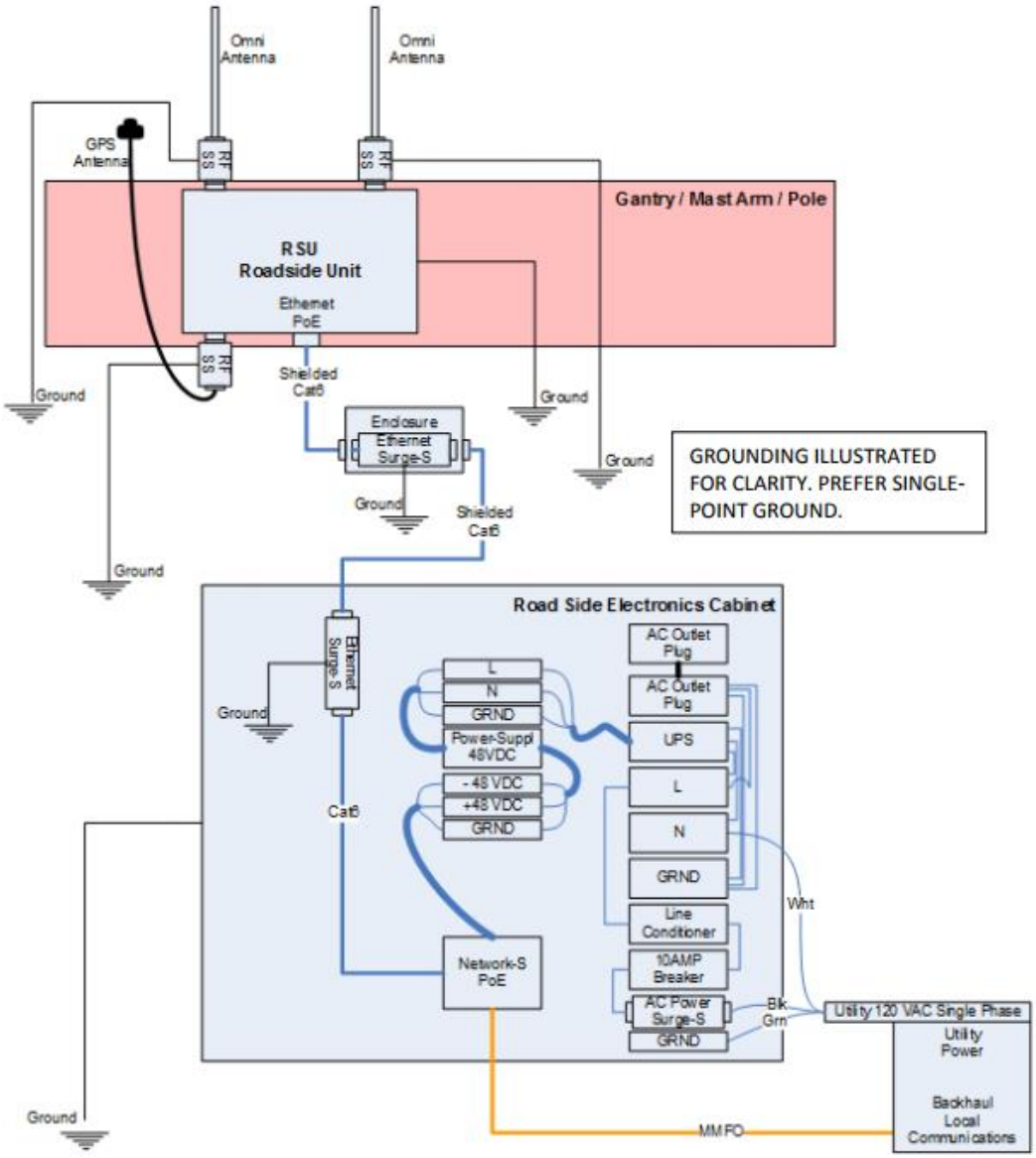


FIGURE 4: POLE MOUNTED RSE

[4]

[4] DSRC Roadside Unit (RSU) Specifications Document v4.1 – Florida Department of Transportation

3.3. HARDWARE CONNECTIONS

For implementation of MMITSS, the signal controller must be connected to MMITSS hardware components. Hardware components are connected using MMITSS Powered-Over-Ethernet (PoE) kit.

Figure 5 shows a component diagram for required hardware connections. Following steps can be followed to prepare the signal controller for the MMITSS implementation.

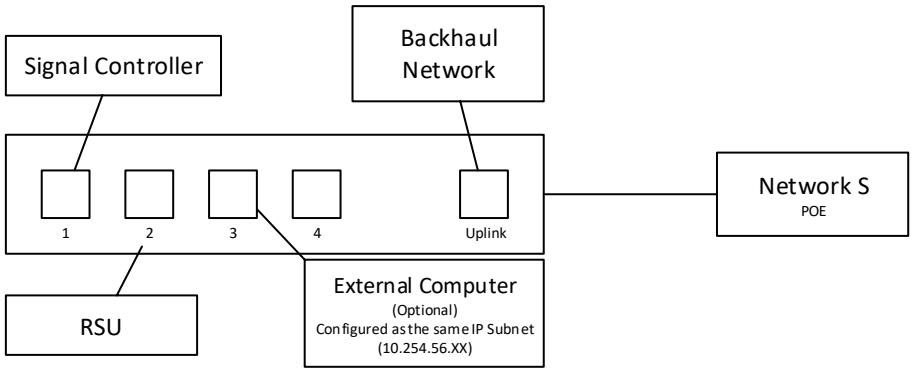
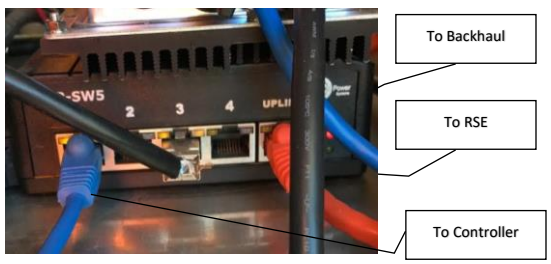
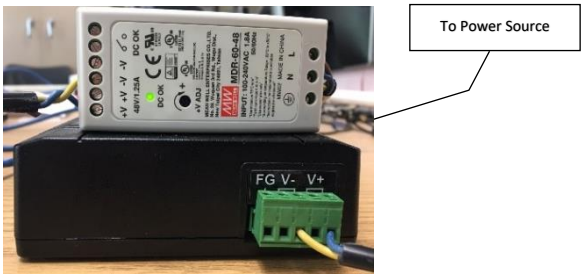


FIGURE 5: POE CONNECTIONS

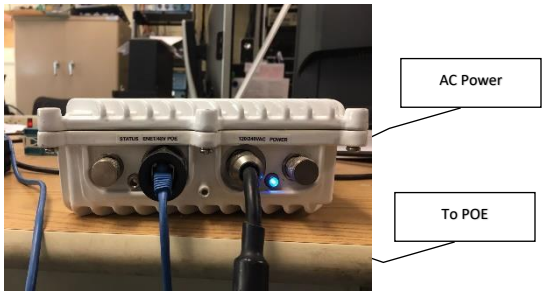
1. Connect UPLINK port of the PoE to the backhaul network.
2. As shown in Figure 5, connect one port of the PoE to the signal controller, one to the RSE, and another to an external computer (optional during operation) which will be used for MMITSS deployment.
3. Figure 6 shows the pictures of the RSU setup on the workbench.



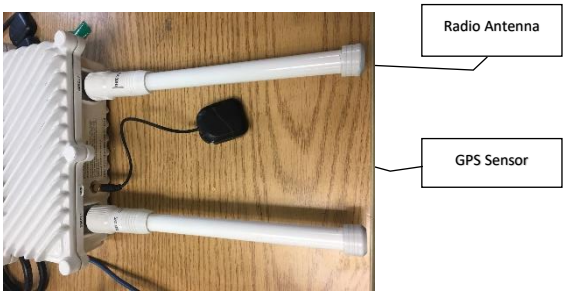
Powered-Over-Ethernet Kit (Front)



Powered-Over-Ethernet Kit (Back)



Road Side Equipment (Front)



Road Side Equipment (Back)

FIGURE 6: POE CONNECTIONS - PICTURES

3.4. RUNNING MMITSS APPLICATIONS ON RSE

Once the RSE is mounted on suitable pole on the intersection, hardware connection links are established and MMITSS applications are installed on the RSU (See Section 3.2), general work flow to run any MMITSS application on the intersection is as follows:

1. Connect the external computer to the PoE kit, as shown in Figure 5.
2. Ensure that the default network gateway, IP address of the external computer, and IP address of the RSE are consistent with each other. For consistency, ensure that the IP addresses are of the form ABC.DEF.GHI.XYZ, where A through I represent the digits that need to be common for all computers/devices in network, whereas X, Y and Z represent the digits which may vary over the devices. For example,
   Default Gateway: 10.254.56.1
   IP Address (Computer): 10.254.56.2
   IP Address (RSE): 10.254.56.3

FIGURE 7: WINSCP

3. Open WinSCP. The icon of WinSCP on the windows desktop is as shown in Figure 7.
4. An sample Login window of WinSCP is shown in Figure 8. In this window,
   - From the drop down menu of *File protocol*, select SCP.
   - In the field *Host name*, enter the IP address of RSE.
   - Leave port number to its default value (22).
   - Use supplier provided (Savari Inc.) credentials in the fields *User name* and *Password*.
   - These details can be saved for later access using the *Save* button.
   - Click *Login* button to enter the communication interface.

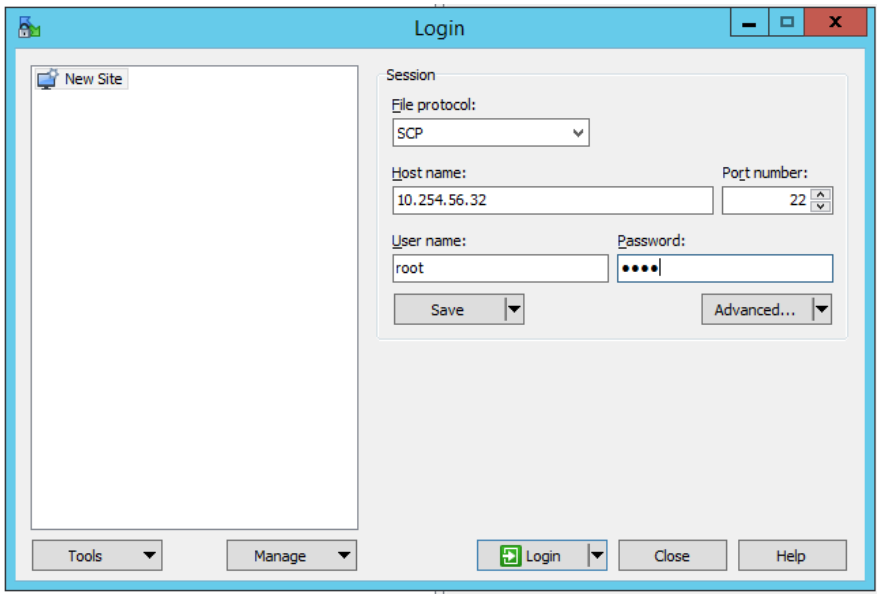


FIGURE 8: LOGIN WINDOW OF WINSCP

   - If the credentials were previously saved, WinSCP may ask for password for confirming credentials, as shown in Figure 9. Enter the supplier provided password to proceed.

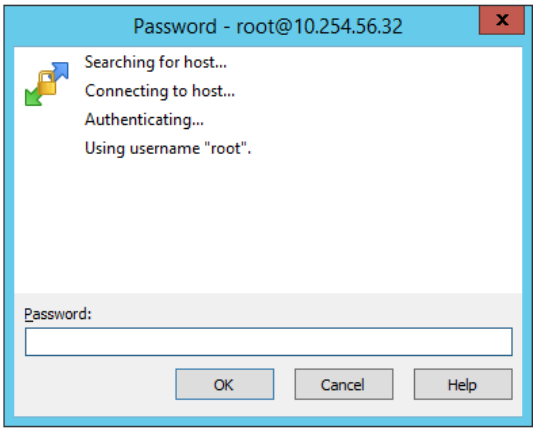


FIGURE 9: CREDENTIALS CONFIRMATION FOR SAVED ACCOUNTS (WINSCP)

5. Upon confirmation of credentials, WinSCP communication link is established and interactive communication interface is displayed, as shown in Figure 10. Through this interface, files or applications in the RSE can be accessed/transferred from the connected external computer.
6. Open PuTTY, which provides a command line interface for running applications on the RSE. Login with the credentials used in Step 4 to proceed.

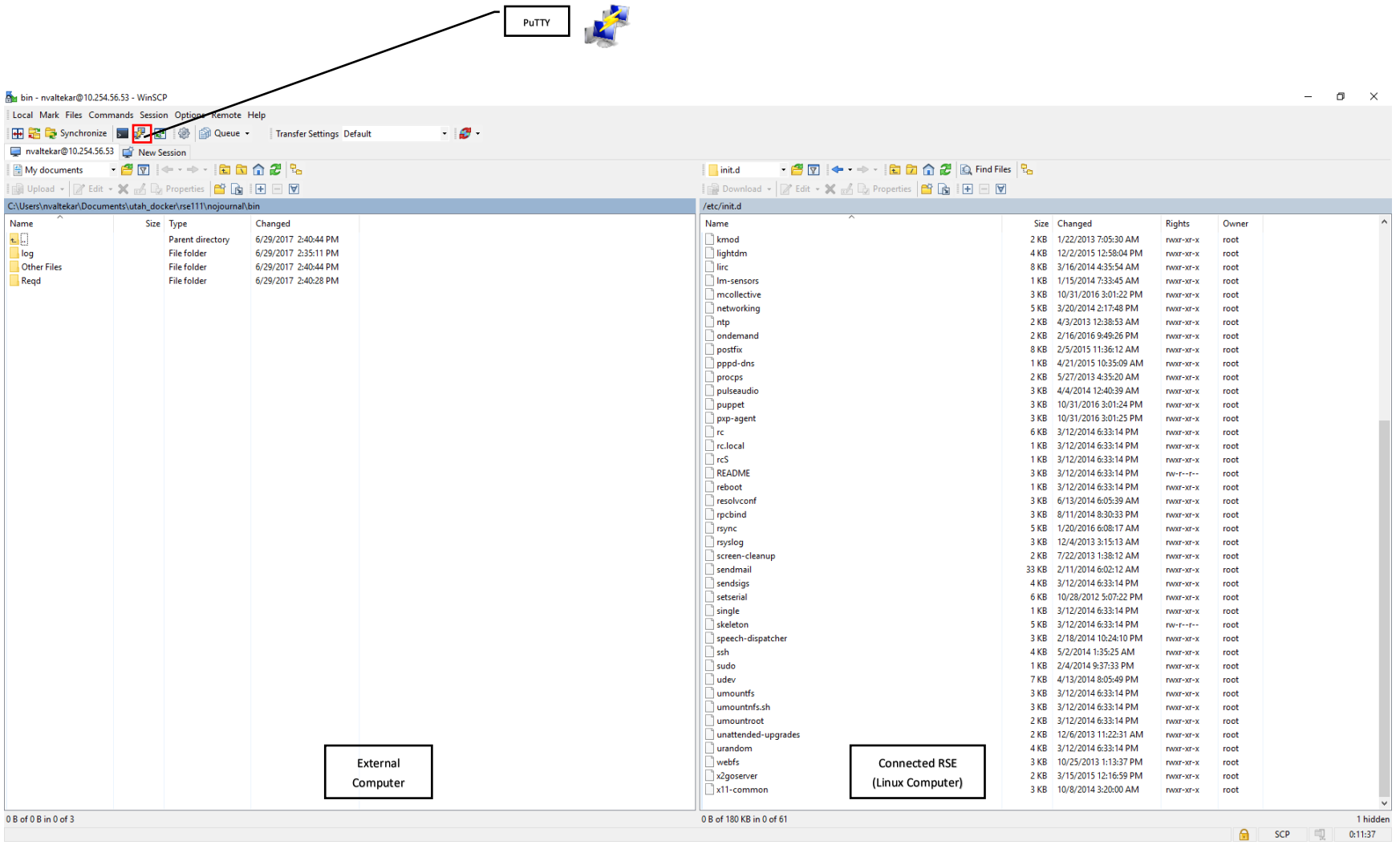


FIGURE 10: WINSCP USER INTERFACE AND LOCATION OF PUTTY

7. Once connected to RSE in the PuTTY, as shown in Figure 11, type following command to access root directory:

```
cd /
```

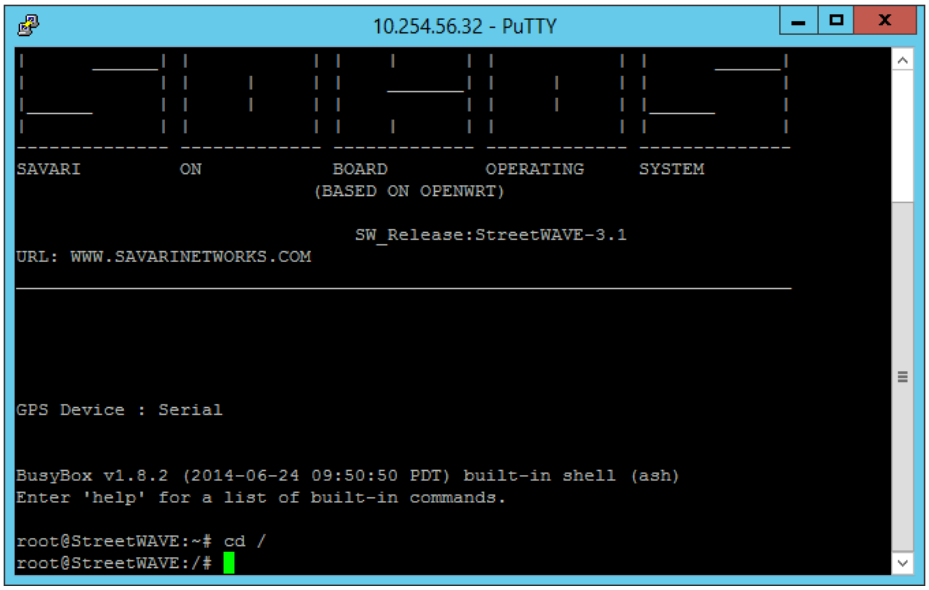


FIGURE 11: PUTTY - ACCESSING ROOT DIRECTORY

8. From root directory, any directory known path can be accessed by typing the path after command cd. For example, for accessing the directory `nojournal/bin/log`, as shown in Figure 12 use,

```
cd nojournal/bin/log/
```

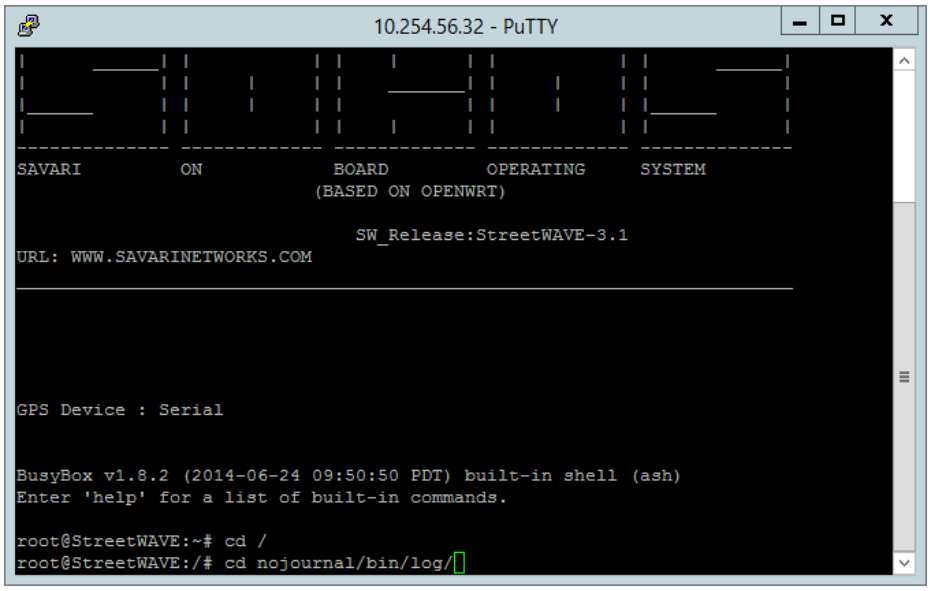


FIGURE 12: PUTTY - ACCESSING ANY KNOWN DIRECTORY

9. *The MMITSS Script* runs all required MMITSS applications. As shown in Figure 13, start the *MMITSS Script* using

```
/etc/init.d/mmitss start
```

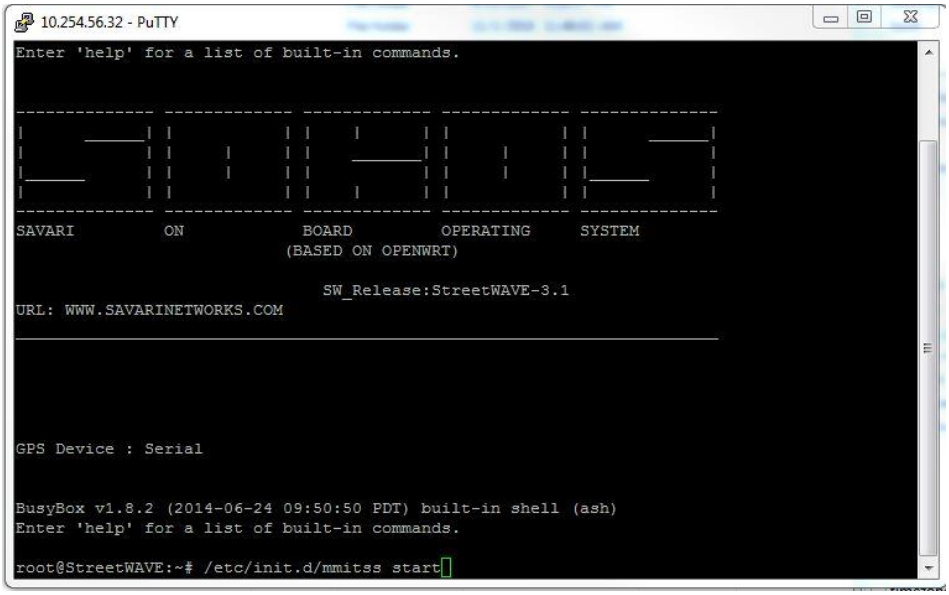


FIGURE 13: PUTTY - STARTING MMITSS SCRIPT

10. As shown in Figure 14, use the following command to stop the *MMITSS Script* and applications:

```
/etc/init.d/mmitss stop
```

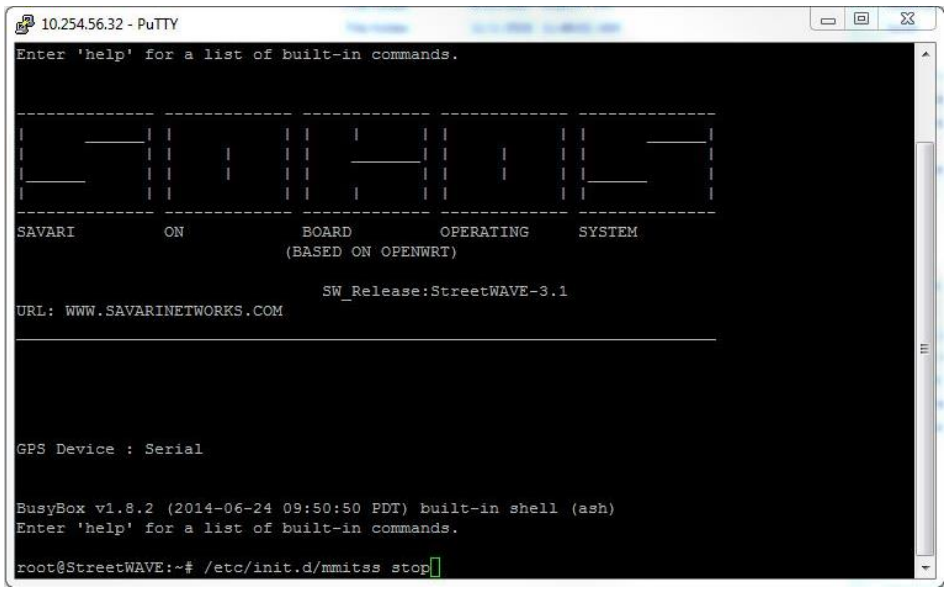


FIGURE 14: PUTTY - STOPPING MMITSS SCRIPT

Note that based on the desired MMITSS component, the *MMITSS Script* may require some modifications such as addition or removal of corresponding MMITSS applications along with their arguments. The MMITSS applications and corresponding configuration files required for each MMITSS component are listed in Section 3.5. An example of the *MMITSS Script* is shown in Figure 15.

```
#!/bin/sh /etc/rc.common
## This is a script to load all the MMITSS applications
## Author : Sriharsha Mucheli
## University of Arizona

START=95

start() {
    /etc/init.d/wmefwd stop
    sleep 1
    /etc/init.d/wmefwd start
    sleep 1
    /etc/init.d/firewall stop
    sleep 1
    /usr/local/bin/MMITSS_MRP_MAP_SPAT_Broadcast 127.0.0.1 127.0.0.1 0 > /nojournal/bin/log/MMITSS_MRP_MAP_SPAT_BROADCAST.log 2>&1 &
    sleep 1
    /usr/local/bin/MMITSS_MRP_PriorityRequestServer_road_final > /nojournal/bin/log/MMITSS_MRP_PriorityRequestServer_road_final.log 2>&1 &
    sleep 1
    /usr/local/bin/MMITSS_MRP_Priority_Solver > /nojournal/bin/log/MMITSS_MRP_Priority_Solver.log  2>&1 &
    sleep 1
    /usr/local/bin/MMITSS_MRP_TrafficControllerInterface_forceoffTest > /nojournal/bin/log/MMITSS_MRP_TrafficControllerInterface_forceofftest.log 2>&1 &
    sleep 1
    /usr/local/bin/MMITSS_rsu_BSM_Receiver 3333 3.6 0 > /nojournal/bin/log/MMITSS_rsu_BSM_Receiver.log 2>&1 &
    sleep 1
    /usr/local/bin/MMITSS_MRP_PerformanceObserver_Field 3333 0.5 10.254.56.199 > /nojournal/bin/log/MMITSS_MRP_PerformanceObserver.log 2>&1 &
    sleep 1
    /usr/local/bin/MMITSS_PED_MAP_BROADCAST 10.254.56.197 > /dev/null 2>/dev/null &
    sleep 1
    /usr/local/bin/MMITSS_RSU_SMARTCROSS 10.254.56.197 > /dev/null 2>/dev/null &
    sleep 1
    /usr/local/bin/MMITSS_MRP_EquippedVRUTrajectory >/dev/null 2>/dev/null &
}
stop() {
    pkill -9 MMITSS
    sleep 1
    /etc/init.d/wmefwd stop
}
```

FIGURE 15: EXAMPLE OF MMITSS SCRIPT

3.5. COMPONENT SPECIFIC MMITSS APPLICATIONS

In order to allow correct implementation of desired MMITSS component, ALL component specific applications along with essential configuration files need to be present in the proper directory in the RSE. In addition, before running the *MMITSS Script*, it needs to be ensured that the *MMITSS Script* contains a starting call to ALL desired MMITSS components.

In the RSE, component specific applications are required to be present at path:

`user/local/bin/`

Whereas, corresponding configuration files are required to be present at path:

`nojournal/bin/`

For each MMITSS component, the required MMITSS applications and configuration are provided in following subsections. For MMITSS applications and its explanation, general syntax followed is as follows:

Application Name Argument 1 Argument 2 Argument 3
Explanation

3.5.1. GENERAL MMITSS APPLICATIONS (APPLICABLE TO ALL COMPONENTS)

3.5.1.1. MMITSS APPLICATIONS

Following applications are general MMITSS applications and are used by all MMITSS components:

i. **`MMITSS_MRP_MAP_SPAT_Broadcast MAP_IP SPAT_IP LOG`**

The MRP_MAP_SPaT_Broadcast component is responsible for constructing and broadcasting J2735 SPAT and MAP data messages. This component receives SPaT data from a signal controller and pack the data into a standard SPaT message. This component reads a map configuration file and packs the data into a standard MAP data message. Arguments for this application are:

a. MAP_IP: Host IP for sending MAP to
b. SPAT_IP: Host IP for sending SPaT to
c. LOG: 1: log incoming SPaT data; 0: don't log SPaT data

ii. **`MMITSS_MRP_EquippedVehicleTrajectoryAware PORT SPEED_CO LOG`**

The MRP_EquippedVehicleTrajectoryAware application is responsible for constructing and storing equipped vehicle trajectories. This component is also responsible for filtering received BSMs based on geo-fencing areas. Based on the received BSMs and the intersection map, this application locates vehicles on the map and stores corresponding information including position, speed, heading, Estimated Time of Arrival (ETA) and requested phase. The trajectory data is used by two other MMITSS components including the MRP_PerformanceObserver and the MRP_TrafficControl component. These components will request trajectory data. Arguments for this application includes:

a. PORT: Specify the number of port that receives BSM, should be 3333 (Default)

b. SPEED_CO: This argument specifies the runtime speed of the application. It should be kept 1.0 to specify real-time operations.
c. LOG: 1- log incoming BSM data; 0- don't log incoming BSM data

iii. MMITSS_MRP_TrafficControllerInterface

The MRP_TrafficControllerInterface application is responsible for providing the protocol specific interface to the traffic signal controller. The MCDOT SMARTDrive network uses Econolite ASC/3 NTCIP controllers with a fiber Ethernet backbone. This application is used for I-SIG and TSP components of MMITSS. The functionalities of this component include:

1. Collect controller's configuration data from traffic signal controller. There are no arguments for this application.
2. Collect vehicle detection data from traffic signal controller.
3. Command MMITSS traffic and priority control to traffic signal controller

There are no arguments for this application.

iv. MMITSS_RSE_Message_Tranceiver

This application is responsible for sending and receiving properly formatted messages over a specified WAVE channel. This application allows other applications to subscribe to send or receive WAVE messages.

One instance of this component is required for each radio/channel/message combination. For example, if the RSE is receiving Signal Request Messages (SRM) on channel 182 and sending Signal Phase and Timing messages (SPaT) on channel 172, this will require two instances of the RSE_Message_Transceiver application to be running with the appropriate configuration files specified as command line arguments. Each of these instances will support the required interfaces for communicating the SRM and SPaT messages to the subscribing components.

Each instance of this application will read the configuration parameters such as radio interface, channel number, message length, remote IP, remote Port and PSID from the wme.conf file. The radio interface and channel number are used to open a WAVE messaging handle with the underlying driver. This handle is then used to implement an event-based mechanism to handle incoming/outgoing messages.

The WAVE Message transceiver works in the same way on both OBE and the RSE. The application is invoked as below:

OBE: `MMITSS_OBE_Message_Tranceiver –f CONF [-d]`

RSE: `MMITSS_RSE_Message_Tranceiver –f CONF [-d]`

Argument: -f specifies which file to use to read configuration information. (e.g. `/etc/config/wmefwd/bsm_wme.conf`)

Extra Argument: -d debug mode to print logs to screen rather than writing to a file.

The instances required for MMITSS applications on RSE are as follows:

For Active Request Table Tranceiver (Active request table is the prototype message used to develop the 2016 J2735 SSM Message)

```
MMITSS_RSE_Message_Tranceiver -f /etc/config/wmefwd/art_wme.conf
```

For BSM Receiver:

```
MMITSS_RSE_Message_Tranceiver -f /etc/config/wmefwd/bsm_wme.conf
```

For MAP and SPaT Transmitter:

```
MMITSS_RSE_Message_Tranceiver -f /etc/config/wmefwd/spat_wme.conf
```

For SRM Receiver:

```
MMITSS_RSE_Message_Tranceiver -f /etc/config/wmefwd/srm_wme.conf
```

An example of bsm_ wme.conf file, used for BSM the receiver is as follows:

```
##########################################
### Sample config file for WME app ###
##########################################
### Configuration enabled: 0 -> Disabled; 1-> Enabled; Default -> 1
Enabled = 1
### Name of the Message: Default -> WME_MSG
MsgName = BSM
### Direction: 0 -> Outgoing; 1 -> Incoming; Default -> 1
Direction = 1
### Channel number to use for WME: Default -> 172
ChannelNumber = 172
### Interface to use for WME: Default -> ath0
InterfaceName = ath1
### PSID of the message which uses this config file: Default -> 0x10
PSID = 0x20
### Channel Mode: 0 -> Alternating; 1 -> Continuous; Default -> 1
ChannelMode = 1
### Remote IP to which socket is opened: Default -> 127.0.0.1
RemoteIP = 127.0.0.1
### Remote port to which socket is opened: Default -> 9999
RemotePort = 3333
### Protocol to use for socket: Default -> UDP
RemoteProtocol = UDP
### Provider Service Context: Default -> "PSC"
PSC = MMITSS
### Message Length: Default -> 128 bytes
MsgLength = 75
### Tx power level : Default -> 15 (max : 33)
TxPower = 22
```

This configuration file shows an example of BSM transceiver in an RSE. For different message types on different devices, users need to modify: Name, Direction, Channel, Interface, PSID, Remote Port, and Message Length as well as the configuration file name (e.g. srm_wme.conf). Table 4 can be used for reference while modifying the configuration

files. All the WAVE message transceiver configuration files should be copied to `/etc/config/wmefwd`

TABLE 4: REFERENCE TO WME CONFIGURATION FILES

Line	Syntax	Defines	Default Values
1	Enabled	Defines if the configuration shall be used: 0-> Disabled 1-> Enabled	1
2	MsgName	The name of the WME. ART, BSM, SPaT, SRM	WME_MSG
3	Direction	The direction of the WME: 0-> Outgoing; 1->Incoming.	1
4	ChannelNumber	The channel number that will be used for WME. ART -> 182 BSM -> 172 SPaT -> 172 SRM -> 182	172
5	InterfaceName	Interface to use for WME ART -> ath0 (Corresponds to channel 182) BSM -> ath1 (Corresponds to channel 172) SPaT -> ath1 (Corresponds to channel 172) SRM -> ath0 (Corresponds to channel 182)	ath0
6	PSID	PSID of the message that uses this config file (As defined in SAE J2735 Standard) ART -> 0x79 BSM -> 0x20 SPaT -> 0x60 SRM -> 0x40	0x10
7	ChannelMode	Channel Mode: 0-> Alternating 1-> Continuous **(Use Continuous)**	1
8	RemoteIP	IP address for socket, for transferring data Use 127.0.0.1	127.0.0.1
9	RemotePort	Port number for socket ART -> 15040 BSM -> 3333 SPaT -> 15030 SRM -> 4444	9999
10	RemoteProtocol	Protocol to use for socket: Use UDP	UDP

11	PSC	Provider Service Context: Use MMITSS	PSC
12	MsgLength	Length of the Message (Length of Source Code) ART -> 300 BSM -> 65 SPaT -> 1000 SRM -> 256	128
13	TxPower	Power of DSRC radios (dB). Can be adjusted to manipulate the physical communication range of the radios. (max:33)	15

3.5.1.2. CONFIGURATION FILES

Following configuration files are general MMITSS configuration files and are required for all MMITSS components.

i. `ConfigInfo.txt`

This file is used for defining the location of ConfigInfo_<IntersectionName>.txt file on the RSU, which provides the inputs to the signal controller related to important timing parameters of traffic signal. Example of content of ConfigInfo.txt file is as follows:

```
/nojournal/bin/ConfigInfo_Dedication.txt
```

This file tells the signal controller that the signal configuration file is located at the given path.

ii. `ConfigInfo_<IntersectionName>.txt` (Updated during the operation)

The content of the file is the signal parameters of the signal controller including number of phases in use, phase sequence, minimum green time, maximum green time, yellow time, red clearance time, pedestrian walking time and pedestrian clearance time. This file needs to be present at the beginning of MMITSS operations and is updated by MMITSS applications during the operation, based on the data read from the signal controller. An example of content of ConfigInfo_<IntersectionName>.txt file is as follows:

```
Phase_Num   4
Phase_Seq   0     2     0     4     5     6     0     0
Gmin        0     15    0     8     4     15    0     0
Yellow      0     4     0     3     3     4     0     0
Red         0     2     0     4.2   1     2     0     0
Gmax        0     55    0     35    8     47    0     0
PedWalk     0     0     0     7     0     7     0     0
PedClr      0     0     0     33    0     20    0     0
```

iii. `nmap_name.txt`

This file specifies the location of map description file (<IntersectionName>.nmap) on the RSE. Example of content of nmap_name.txt file is as follows:

```
/nojournal/bin/Daisy_Dedication.nmap
```

iv. **<IntersectionName>.nmap**

The content of the file is the intersection map description file. Details of how to construct the map and an example map file can be found in Appendix-I.

v. **IPInfo.txt**

The content of the file is the IP address and port information for the signal controller. The example of IPInfo.txt file is as follows:

```
10.254.56.12
501
```

First line of the file indicates the IP address of the signal controller whereas the second line provides its port number.

vi. **ntcipIP.txt**

The content of this file is the same that of IPInfo.txt file.

vii. **Lane_Phase_mapping.txt**

The content of the file is the phase to lane mapping in order to calculate the requested phase from messages that are not aware of phases. The value is the phase number. The sequence of the numbers in line 2 is phase of approach 1 through, approach 1 left, approach 3 through, approach 3 left, approach 5 through, approach 5 left, approach 7 through, approach 7 left. The approach number should match the map of intersection. If a phase doesn't exist, put zero, if left turn and through share the same phase, put the same value. For the example of intersection configuration shown in Figure 16, the content of Lane_phase_mapping file shall be as follows:

```
#The order of the approach 1,3,5,7 through left. If
the phase doesn't exist, then put phase 0

6 1 8 3 2 5 4 7
```

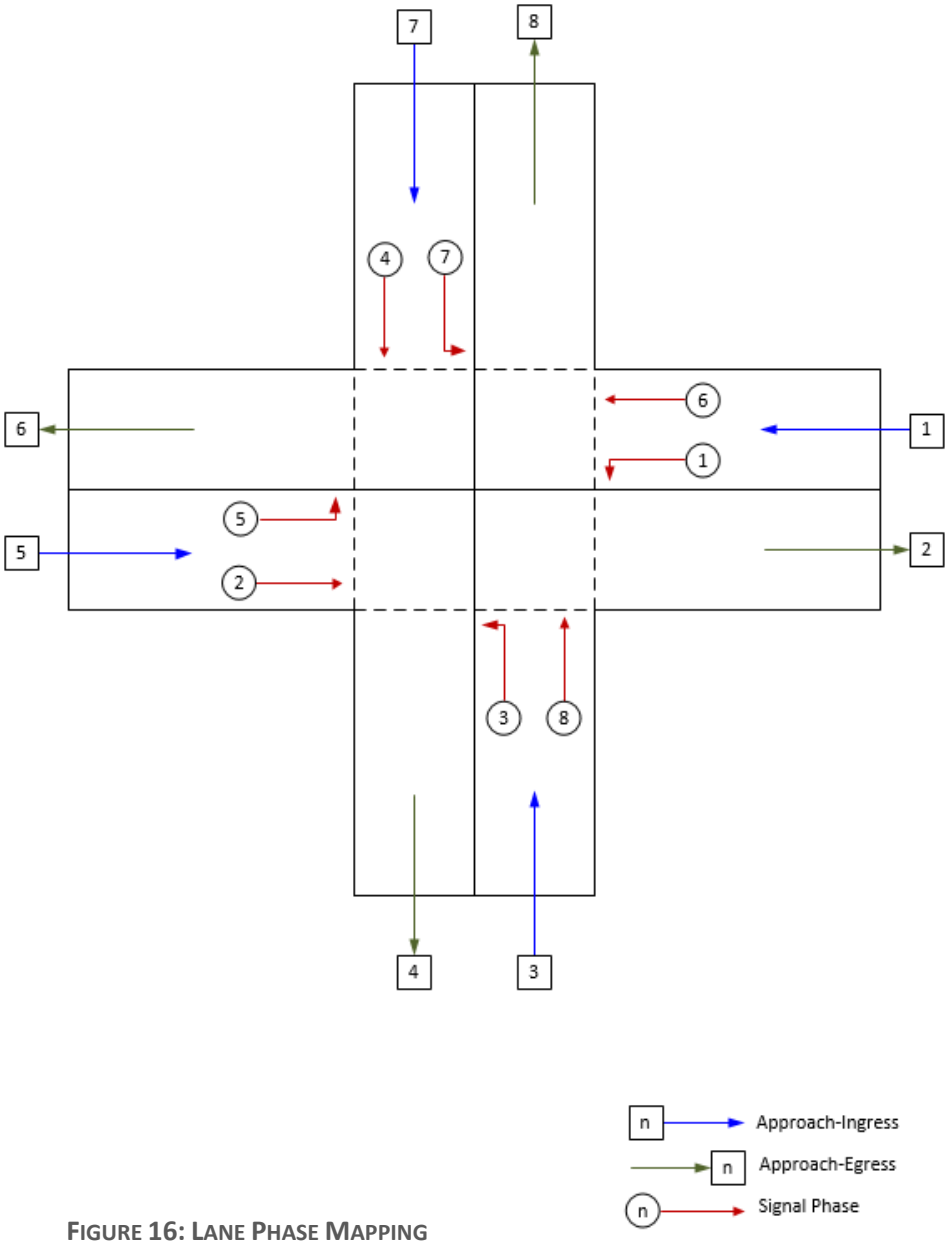


FIGURE 16: LANE PHASE MAPPING

3.5.2. INTELLIGENT SIGNAL CONTROL (I-SIG)

3.5.2.1. MMITSS APPLICATIONS

i. MMITSS_MRP_Signal_control PORT PR OBJ

The MRP_Signal_control application is responsible for providing adaptive signal control for all non-priority vehicles at an intersection. MRP_Signal_control integrates information from connected vehicles and traditional vehicle detectors. MRP_Signal_control performs detection matching, phase calls, phase extensions, dilemma zone protection for each of the different modes (and dynamics) of vehicles. Arguments for this application are:

a. PORT: Port where vehicle trajectory is requested from VehicleTrajectoryAware : should be 3333.
b. PR: Market penetration rate (change when needed).
c. OBJ: 0 - objective is to minimize total vehicle delay; 1- objective is to minimize total queue length.

3.5.2.2. CONFIGURATION FILES

i. Signal_Config_COP.txt (Auto-Generated)

This file is auto-generated during the MMITSS operations. The content of the file is similar the ConfigInfo_<IntersectionName>.txt file which are signal parameters of the signal controller including number of phases in use, phase sequence, minimum green time, maximum green time, yellow time, red clearance time, pedestrian walking time and pedestrian clearance time. A sample of this file is as follows:

```
Phase_Num 8
Phase_Seq 1 2 3 4 5 6 7 8
Gmin  7     7     7     7     7     7     7     7
Yellow      3     3     3     3     3     3     3     3
Red   2     2     2     2     2     2     2     2
Gmax  35    35    35    35    35    35    35    35
PedWalk 0   5     0     5     0     5     0     5
PedClr      0     25    0     25    0     25    0     25
```

ii. DSRC_Range.txt (Auto-Generated)

This file is auto-generated during the MMITSS operations. The content of the file is the geo-fencing area of the intersection which specifies the length of the extension of map nodes from the reference point for each leg of the intersection in the first line where the order of the approaches is from 1-8. 1:SB Approaching; 2:NB Leaving; 3:WB Approaching; 4:EB Leaving; 5:NB Approaching; 6:SB Leaving; 7: EB Approaching; 8:WB Leaving. The second line indicates the width and height

of the rectangle in the middle of the intersection. A sample of this file is as follows:

```
#The order of the approaches is from 1-8. 1:SB Approaching; 2:NB Leaving; 3:WB Approaching; 4:EB Leaving; 5:NB Approaching; 6:SB Leaving; 7: EB Approaching; 8:WB Leaving

198 101 275 105 242 241 273 276

#width and Height of the rectangle in the middle of intersection
```

3.5.3. TRAFFIC SIGNAL PRIORITY (TSP)

3.5.3.1. MMITSS APPLICATIONS

i. MMITSS_MRP_Priority_Solver –s NETWORK –c INTEGRATION

The MRP_Priority_Solver is responsible for scheduling the priority requests in the Active Request Table (ART) from multiple priority eligible vehicles on the same or conflicting movements and providing the best signal timing to serve all of the requests simultaneously with the consideration of the prevailing traffic conditions and the requested level of priority (as determined by the vehicle and the established policy). In addition, data representing the signal controller configuration, i.e., minimum green, maximum green, red and yellow clearance intervals is required to determine the optimal priority signal timing. The output of the optimization solver is an optimal phase timing schedule that accommodates all active requests in the ART. The optimal phase timing schedule is provided to the MRP_TrafficControllerInterface.

a. NETWORK: -s 1: for congested network; –s 0: for regular network
b. INTEGRATION: -c 1: for using signal priority control with actuated control; -c 2: for using signal priority control with I-SIG.

ii. MMITSS_MRP_PriorityRequestServer_field –o COOR –c INTEGRATION

The MRP_PRS_PriorityRequestServer application is responsible for managing and prioritizing the priority requests (SRM) from multiple priority eligible vehicles on the same or conflicting movements with the consideration of the prevailing traffic conditions and the requested level of priority (as determined by the vehicle and the established policy). This application constructs an Active Request Table (ART) based on received SRMs and passes the ART as input to MRP_Priority_Solver application. The Active Request Table (ART) is a list of priority requests that are currently active. Arguments for this application are:

a. COOR: -o 1: for considering coordination as a form of priority; –o 0: without considering coordination.

b. INTEGRATION: -c 1: for using signal priority control with actuated control; -c 2: for using signal priority control with I-SIG.

3.5.3.2. CONFIGURATION FILES

i. RSEid.txt

This file contains the name of the intersection. Example of content is as follows:

```
Daisy_Dedication
```

ii. priorityConfiguration.txt

This file sets the priority configuration and coordination settings by assigning priority weights to different vehicle modes. Example of content of this file is as follows:

```
coordination 0
coordinated_phase 2 6
cycle 70
offset 0
split 10
transit_weight 1
truck_weight 50
```

iii. InLane_OutLane_Phase_Mapping.txt (Auto-Generated).

This file is auto-generated during the MMITSS operations. The content of the file is the inlane / outlane mapping to phases in order to calculate requested phase. This file is generated by MMITSS_MRP_MAP_SPAT_Broadcast. The example of content of this file is as follows:

```
IntersectionID    301
No_Approach 8
No_Phase    8
No_Ingress  4
Approach    1
No_Lane     3
1.1 6.1 4 ## Ingress_Lane Connected Egress_Lane Signal_Phase
1.2 6.1 4
1.3 4.1 7
end_Approach
Approach    3
No_Lane     4
3.2 8.1 6
3.3 8.1 6
3.4 8.1 6
3.5 6.1 1
end_Approach
Approach    5
No_Lane     3
5.2 2.1 8
5.3 2.1 8
5.4 8.1 3
end_Approach
Approach    7
No_Lane     4
7.2 4.1 2
7.3 4.1 2
7.4 4.1 2
7.5 2.1 5
end_Approach
end_file
```

iv. requests.txt (Auto-Generated).

This file is auto-generated during the MMITSS operations. The MRP_PRS_PriorityRequestServer writes ART into a file called "requests.txt". Whenever a new SRM is received, MRP_PRS_PriorityRequestServer updates the contents of this file. Along with ART, MRP_PRS_PriorityRequestServer changes a flag in this file to a positive number whenever a new SRM is received and the ART is updated. The flag is an indicator for the MRP_Priority_Solver component. The MRP_Priority_Solver continuously reads this file and if the flag is positive, the MRP_Priority_Solver solves an optimization problem and changes the flag back to zero. The example of this file is as follows:

```
Num_req 1 1
115 2 14 4 0 1428888550 11 61 1 21 53 1 22 7 1 0
```

In this example, the first number in the first row shows the total number of requests. The second number is the flag that is 1 in this example. The second row shows the request information.

i. **requests_combined.txt (Auto-Generated).**

This file is auto-generated during the MMITSS operations. An example of request_combined.txt file is similar to that of requests.txt, however, requests from EVs are logged into the requests_combined.txt along with requests from other modes, whereas requests.txt contains the requests from all modes except EVs.

ii. **Results.txt (Auto-Generated).**

This file is auto-generated during the MMITSS operations. The content includes the solution of the optimizer whenever the solver is called. An example of this file is as follows:

```
  2    6
 0.00   0.00 26.89  16.46
 0.00  16.00   0.00   0.00   0.00  16.00   0.00   0.00
 0.00  21.50   0.00   0.00   0.00  21.50   0.00   0.00
 0.00   9.50   0.00   0.00   0.00   9.50   0.00   0.00
 0.00  15.00   0.00   0.00   0.00  15.00   0.00   0.00
  1
6   2.50   9.50   0.00 1
 0.00
```

iii. **signal_status.txt (Auto-Generated).**

This file is auto-generated during the MMITSS operations. The content includes 8 numbers that show the status of each phase. An example of this file is as follows:

```
301 signal_status 1 3 1 1 1 3 1 1
```

The intersection ID in this example is 301. Digit 3 indicates the phase is green, digit 1 indicates it is red, and digit 4 indicates it is yellow. The signal is currently on green for phase 2 and 6.

iv. **NewModel.mod (Auto-Generated).**

This file is auto-generated during the MMITSS operations. The file includes the mathematical model that will be used by optimizer (GLPK).

v. **NewModel_EV.mod (Auto-Generated).**

This file is auto-generated during the MMITSS operations. The file includes the mathematical model that will be used by optimizer (GLPK) whenever there is an EV at the intersection.

vi. **NewModelData.dat (Auto-Generated).**

This file is auto-generated during the MMITSS operations. The file contains the data representing signal controller configuration and serves as an input for the mathematical model. An example of this file is as follows:

```
data;
param current:=0;
param SP1:=2;
param SP2:=6;
param init1:=0;
param init2:=0;
param Grn1 :=26.8947;
param Grn2 :=16.4616;
param y           :=    2  4  6  4;
param red         :=    2  2.5     6  2.5;
param gmin        :=    2  15 6  15;
param gmax        :=    2  50 6  50;

param priorityType:= 1 1 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9 0
10 0  ;
param PrioWeigth:=  1 1 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9 0 10
0 ;
param Rl (tr): 1 2 3 4 5 6 7 8:=
1  .  .  .  .  .  2.5  .  .  ;
param Ru (tr): 1 2 3 4 5 6 7 8:=
1  .  .  .  .  .  9.5  .  .  ;
end;
```

vii. Configinfo_EV.txt (Auto-Generated)

This file is auto-generated during the MMITSS operations. It is generated by MMITSS_MRP_Priority_Solver. The file is used whenever there is an EV in the active request table. A sample of content of this file is as follows:

```
Phase_Num 2
Phase_Seq 0 2 0 0 0 6 0 0
Gmin   0     15    0     0     0     15    0     0
Yellow       0     4     0     0     0     4     0     0
Red    0     2.5   0     0     0     2.5   0     0
Gmax   0     35    0     0     0     40    0     0
```

3.5.4. Real-Time Performance Observer (PERF-OBS)

3.5.4.1. MMITSS APPLICATIONS:

i. MMITSS_MRP_PerformanceObserver_Field PORT PR SERVER_IP

The main responsibility of the MRP_PerformanceObserver is to acquire data from other MRP components, including the MRP_EquippedVehicleTrajectoryAware and the MRP_TrafficControllerInterface components, to compute observed performance measures. The MRP_PerformanceObserver includes estimation, managing, and archiving the data. All the acquired performance observations and

metrics are collected and reported in terms of peak period, all day, or specially defined time period. Arguments for this application are:

i. PORT: Port that request vehicle trajectory from VehicleTrajectoryAware
ii. PR: Market penetration rate (change when needed)
iii. SERVER_IP: IP Address of the Database Server Machine.

3.5.4.2. CONFIGURATION FILES

i. GeoFence_Range.txt

The content of the file is the geo-fencing area of the intersection which specifies the length of the extension of map nodes from the reference point for each leg of the intersection. Example of the file is as follows:

```
#The order of the approaches is from 1-8.
1:SB Approaching; 2:NB Leaving; 3:WB Approaching;
4:EB Leaving; 5:NB Approaching; 6:SB Leaving; 7: EB
Approaching; 8:WB Leaving
400 400 400 400 400 400 400 400
#Distance to the nearest line to be considered out
of the Geo Fence Area in meter
8
```

ii. Lane_Movement_Mapping.txt

The content of the file is the movement mapping to lane in order to do the performance observation by movement. The value is the lane number. The sequence of the number in line 2 is lane number of approach 1 right, approach 1 left, approach 3 right, approach 3 left, approach 5 right, approach 5 left, approach 7 right, approach 7 left. The approach numbers should match the map of intersection. If a designated left-turn or right-turn lane doesn't exist, put zero. The sequence of the numbers on line 4 is the arbitrary numbers assigned to each travel time section at intersection. This could be used for comparison and data validation in simulation to be matched with VISSIM travel time sections. For an example network shown in Figure 17, the content of this file is as follows:

```
#The order of the approaches would be 1,3,5,7. IF
doesn't exist, put 0. The first value is Right Turn
Lane, the second value is the Left Turn Lane for
each approach: SBRT SBLT WBRT WBLT NBRT NBLT EBRT
EBLT
1 3 1 3 1 3 1 3
#Movements in this order for travel time sections:
NB NBRT NBLT WB WBRT WBLT SB SBRT SBLT EB EBRT EBLT
43 45 44 46 48 47 49 51 50 52 54 53
```

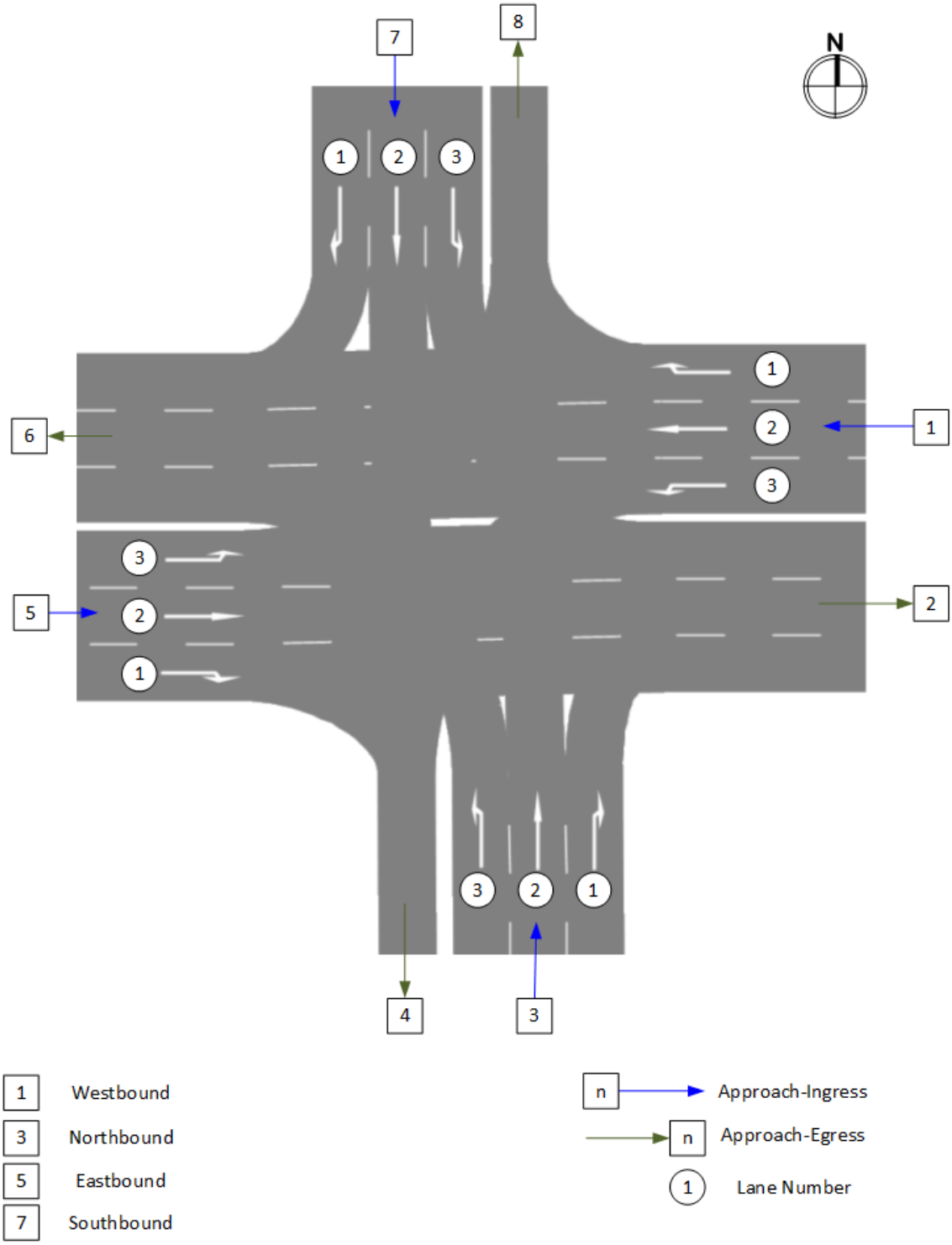


FIGURE 17: SAMPLE NETWORK FOR LANE MOVEMENT MAPPING

3.5.5. MOBILE ACCESSIBLE PEDESTRIAN SIGNAL SYSTEM (PED-SIG)

3.5.5.1. MMITSS APPLICATIONS

i. **`MMITSS_Ped_Map_Broadcast PHONE_IP`**

MRP_Ped_MAP_Broadcast broadcasts the crosswalk map that is received on the smartphone by the Nomadic_MMITSSApp. This map is then decoded on the smartphone by a class based on the pedestrian's GPS location can recognize if it is located near crosswalk. The argument for this application is:

PHONE_IP: IP address of the smartphone

ii. **`MMITSS_MRP_PedRequestServer PHONE_IP`**

The MRP_PedRequestServer is responsible for reading the Ped SPaT status from MRP_MAP_SPAT_Broadcast and broadcasting it to the smartphone. It also receives the Ped calls from the smartphone and place those calls on the traffic controller. Argument for this application is:

PHONE_IP: IP address of the smartphone.

3.5.5.2. CONFIGURATION FILES

There are no component specific configuration files required on the RSE for the PED-SIG component.

3.5.5.3. INTEGRATION OF USER SMARTPHONE WITH MMITSS

In addition to above applications, MMITSS Ped Application needs to be installed in the smartphone of the user. The communication between RSE and the smartphone is through WiFi. A router should configured and connected to the RSE in the same subnetwork as shown in the Figure 18.

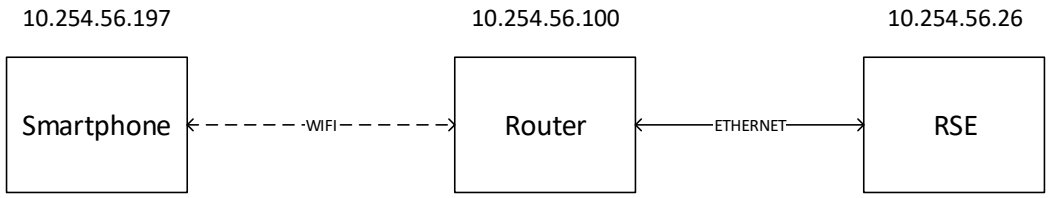


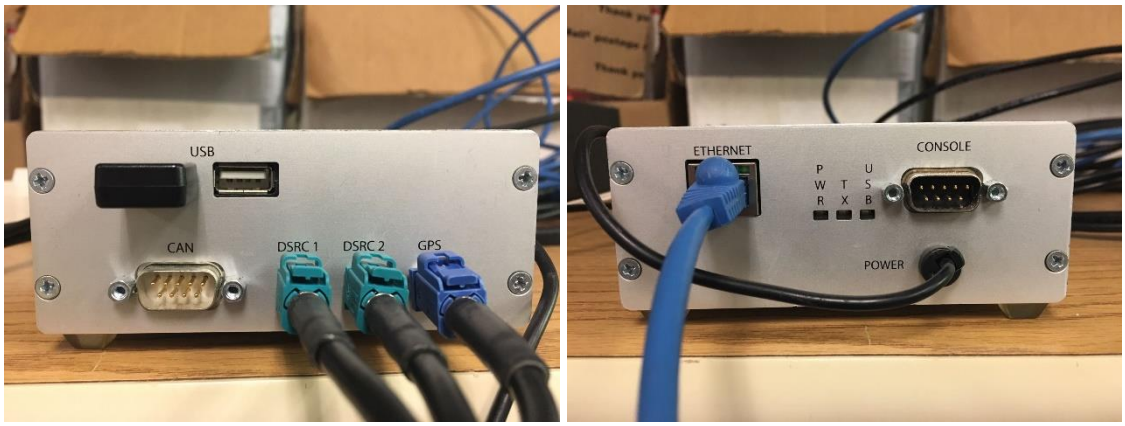
FIGURE 18: COMMUNICATION ARCHITECTURE FOR MMITSS PEDESTRIAN APPLICATION

4. MMITSS DEPLOYMENT ON ON-BOARD EQUIPMENT (OBE)

4.1. PRE-REQUISITES

Prior to running MMITSS applications on the OBE, this user manual assumes following pre-requisites are met:

1. Two DSRC Antennae are connected to the OBE and are ready for communication as shown in Figure 19(A).
2. GPS Antenna is connected to the OBE and is ready for communication as shown in Figure 19(A).
3. OBE has an Ethernet port, with known IP address, which can be used to connect an external computer for device control, as shown in Figure 19(B).
4. OBE is physically installed in the vehicle and is powered on as shown in Figure 19(B).



(A) OBE-DSRC and GPS Connections | (B) OBE-Ethernet and Power Connections

FIGURE 19: OBE - CONNECTIONS

4.2. RUNNING MMITSS APPLICATIONS ON OBE

Once the pre-requisites are met, following steps can be followed to run MMITSS applications on the OBE.

1. Connect the external computer to the OBE via Ethernet.
2. Set the LAN connection settings such that IP address of the external computer is automatically assigned.
3. Open WinSCP. The icon of WinSCP in windows, is shown in Figure 20.
4. An example of Login window of WinSCP is shown in Figure 21. Here,
   - From the drop down menu of *File protocol*, select SCP.
   - In the field *Host name*, enter the IP address of OBE.
   - Leave port number to its default value (22).
   - Use supplier provided (Savari Inc.) credentials in the fields *User name* and *Password*.
   - Above details can be saved for later access using the *Save* button.

FIGURE 20: WINSCP

- Click *Login* button to enter the communication interface.

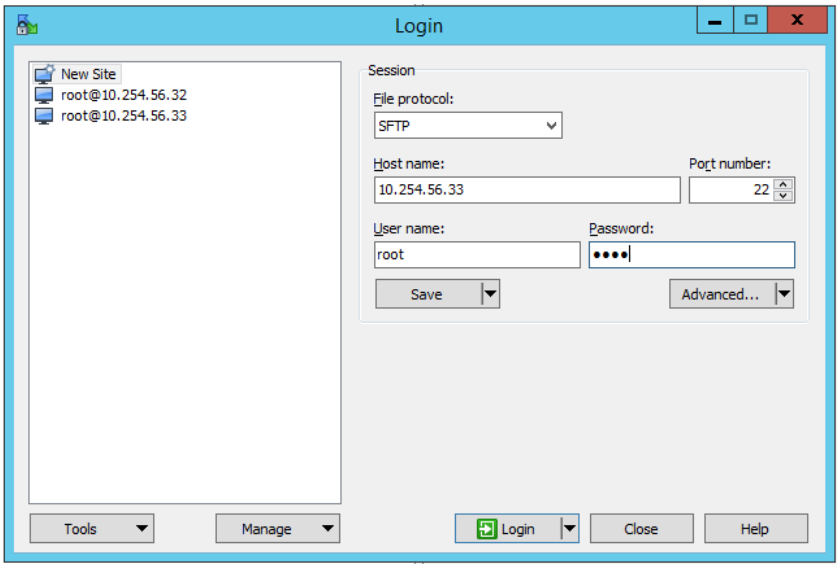


FIGURE 21: WINSCP LOGIN WINDOW

- If the credentials were previously saved, WinSCP may ask for password for confirming credentials, as shown in Figure 22. Enter the supplier provided password to proceed.

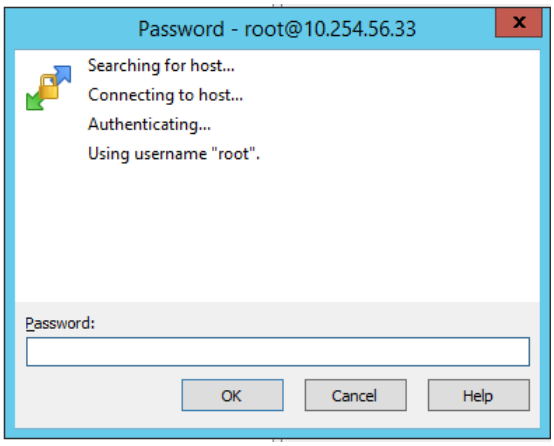


FIGURE 22: WINSCP CREDENTIALS CONFIRMATION FOR SAVED ACCOUNTS

5. Upon confirmation of credentials, WinSCP communication link is established and interactive communication interface is displayed, as shown in Figure 24. Through this interface, files or applications in the OBE can be accessed/transferred from the connected external computer.

6. Open PuTTY, which provides a command line interface for running applications on the OBE. Login with credentials used in Step 4 to proceed, as shown in Figure 23.

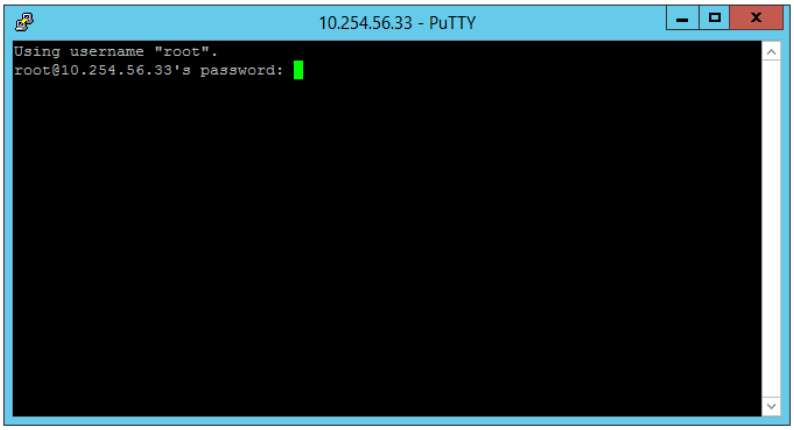


FIGURE 23: ENTERING PUTTY

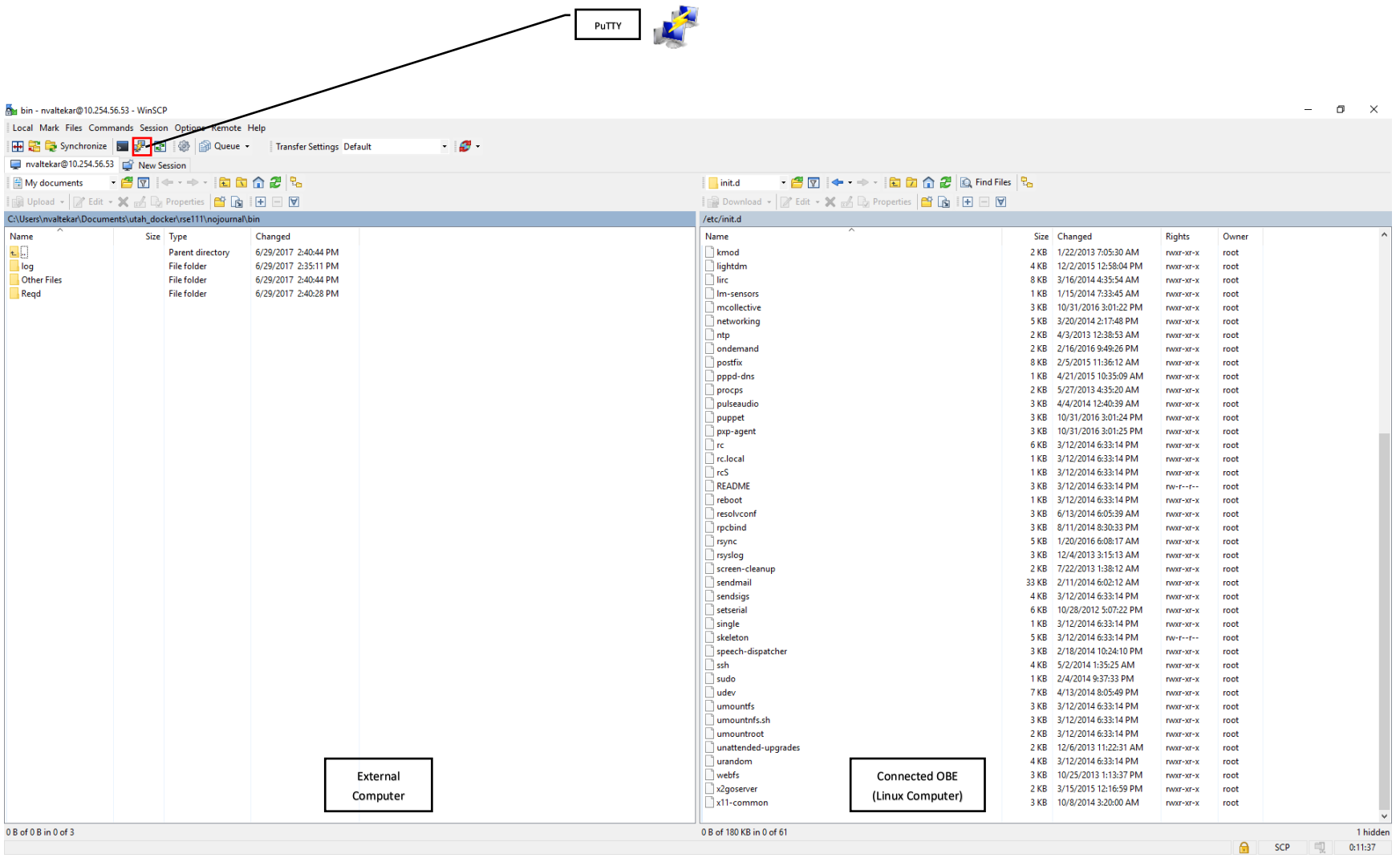


FIGURE 24: WINSCP USER INTERFACE

7. Once connected to RSE in the PuTTY, as shown in Figure 25, type following command to access root directory:

```
cd /
```

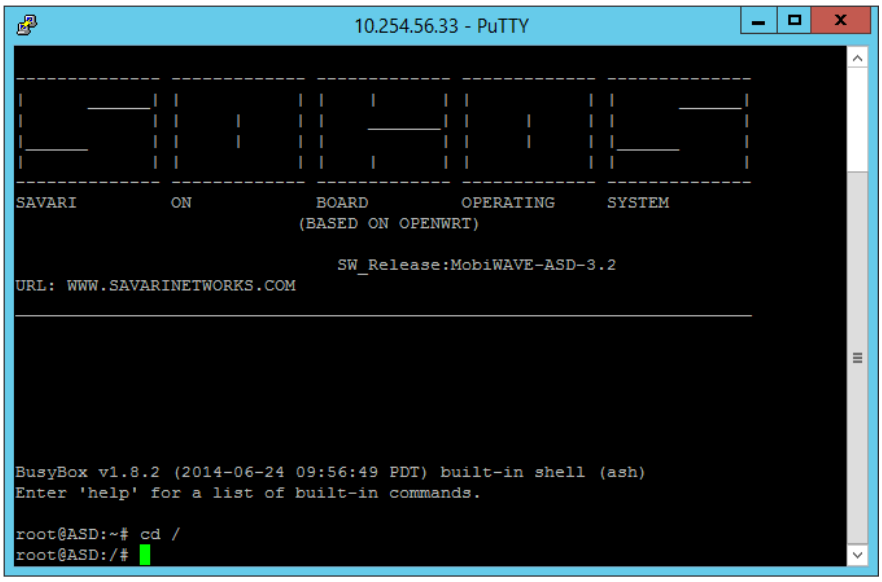


FIGURE 25: PUTTY - ACCESSING ROOT DIRECTORY

8. From root directory, any directory known path can be accessed by typing the path after command `cd`. For example, for accessing the directory `nojournal/bin/log`, as shown in Figure 26, use

```
cd nojournal/bin/log/
```

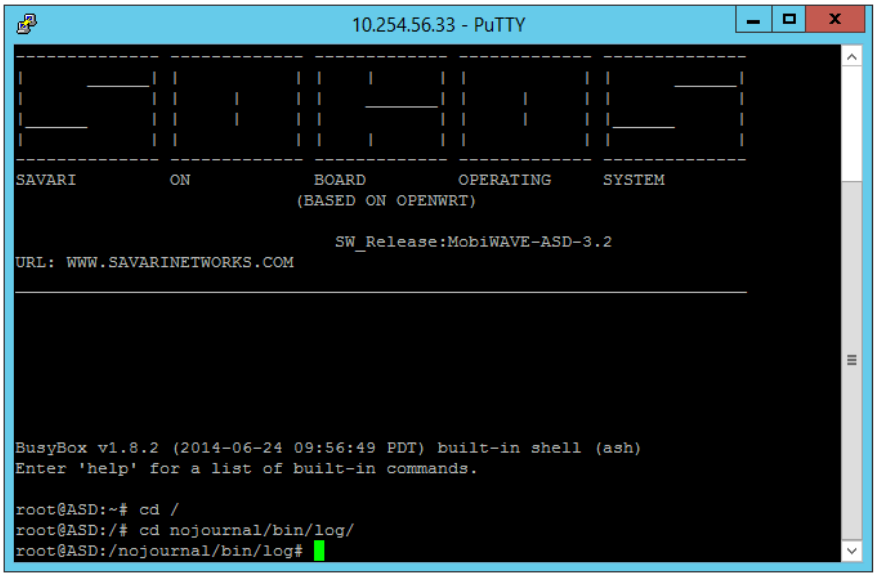


FIGURE 26: PUTTY - ACCESSING ANY KNOWN DIRECTORY

9. *The MMITSS Script* runs all required MMITSS applications. As shown in Figure 27, start the *MMITSS Script* using

```
/etc/init.d/MMITSS start
```

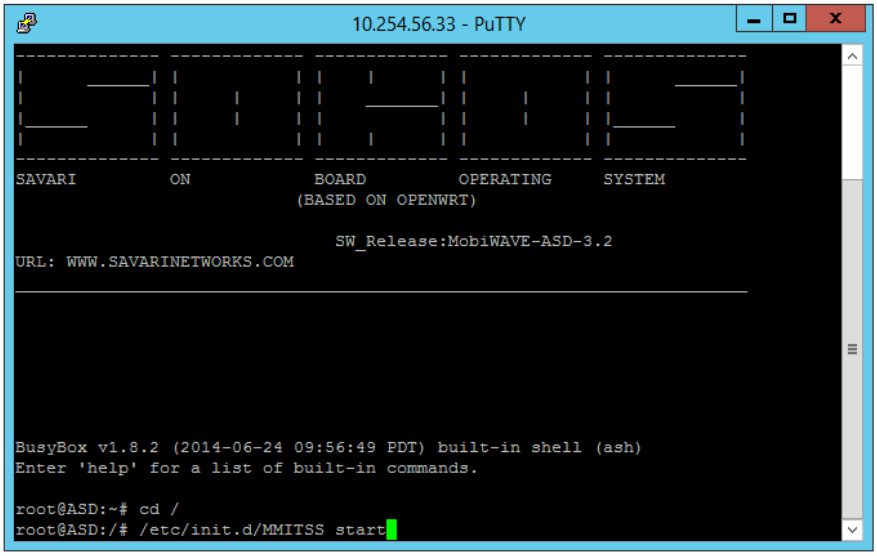


FIGURE 27: PUTTY - STARTING MMITSS SCRIPT

10. To stop all MMITSS applications, as shown in Figure 28, stop the *MMITSS Script* using

```
/etc/init.d/MMITSS stop
```

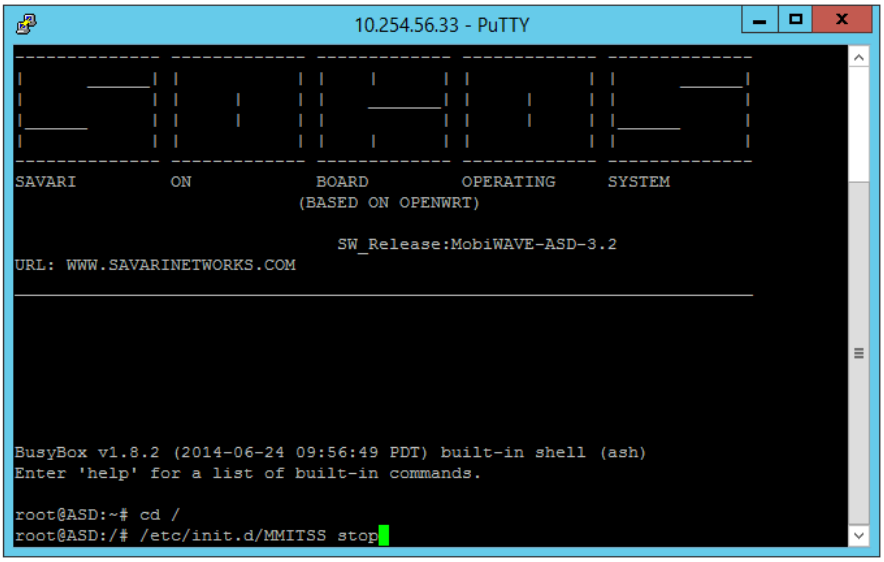


FIGURE 28: PUTTY - STOPPING MMITSS SCRIPT

Note that based on the desired MMITSS component, the *MMITSS Script* may require some modifications such as addition or removal of corresponding MMITSS applications along with their arguments. The MMITSS applications and corresponding configuration files required for each MMITSS component are listed in Section 4.5. An example of the *MMITSS Script* on the OBE is shown in Figure 29.

```
## MMITSS Project
## University of Arizona

START=99

start () {
        echo "Stopping Firewall..."
        /etc/init.d/firewall stop
        sleep 1

        echo "Starting Wave Forwarders..."
        /etc/init.d/wmefwd start
        sleep 1

        echo "Starting bsmd..."
        /usr/local/bin/BSMd > /dev/null 2>/dev/null &
        sleep 1

        echo "Starting MMITSS_OBE_MAP_SPAT_Receiver..."
        /usr/local/bin/MMITSS_OBE_MAP_SPAT_Receiver > /dev/null 2>/dev/null &
        sleep 1

        echo "Starting MMITSS_OBE_PriorityRequestGenerator_Road_..."
        /usr/local/bin/MMITSS_OBE_PriorityRequestGenerator_Road > /dev/null 2>/dev/null &
        sleep 1

        echo "Starting MMITSS_OBE_Accident_Receiver..."
        /usr/local/bin/MMITSS_OBE_Accident_Receiver > /dev/null 2>/dev/null &
        sleep 1

        echo "Starting MMITSS_obu_BSM_Receiver..."
        /usr/local/bin/MMITSS_obu_BSM_Receiver 3333 3.6 1 > /dev/null 2>/dev/null &
        sleep 1

        echo "Starting MMITSS_rsu_BSM_Receiver..."
        /usr/local/bin/MMITSS_rsu_BSM_Receiver 1133 3.6 1 > /dev/null 2>/dev/null &
        sleep 1

        echo "Starting MMITSS_OBE_SZA_Receiver..."
        /usr/local/bin/MMITSS_OBE_SZA_Receiver 127.0.0.1 > /dev/null 2>/dev/null &
        sleep 1

        echo "Ready to Roll"
}
```

FIGURE 29: EXAMPLE OF MMITSS SCRIPT

4.3. COMPONENT SPECIFIC MMITSS APPLICATIONS

In order to allow correct implementation of desired MMITSS component, ALL component specific applications along with essential configuration files need to be present in the OBE. In addition, before starting the *MMITSS Script*, it needs to be ensured that the *MMITSS Script* contains a starting call to ALL component specific MMITSS applications.

In the OBE, component specific applications are required to be present at path:

`user/local/bin/`

Whereas, corresponding configuration files are required to be present at path:

`nojournal/bin/`

For each MMITSS component, the required MMITSS applications and configuration are provided in following subsections. For MMITSS applications and its explanation, general syntax followed is as follows:

Application Name Argument 1 Argument 2 Argument 3

Explanation

4.3.1. GENERAL MMITSS APPLICATIONS

4.3.1.1. MMITSS APPLICATIONS

i. **MMITSS_OBE_BSMData_Transmitter**

The OBE_BSMData_Transmitter is responsible for collecting vehicle data from the GPS receiver and, if available, vehicle data systems (CAN bus) and formatting the data into a J2735 BSM Message. This component is responsible for getting vehicle data, GPS data, and forming the BSM message. (Part 1 and Part 2 as data is available). The OBE_BSMData_Transmitter component is responsible for broadcasting the data over the specified radio and channel (ath1 channel 172). There are no arguments for this applications.

ii. **MMITSS_OBE_MAP_SPAT_Receiver**

The OBE_MAP_SPaT_Receiver is responsible for listening for the MAP and SPaT messages and making the received data available to the other OBE components. The MAP and SPaT data are critical to the vehicle component related to requesting priority. There are no arguments for this application.

iii. **MMITSS_OBE_Message_Tranceiver -f /etc/config/wmefwd/bsm_wme.conf**

This component works in the similar way as that of MMITSS_RSE_Message_Tranceiver. For development of WME_Config files, refer Section-3.5.1.1.

4.3.2. TRAFFIC SIGNAL PRIORITY (TSP)

4.3.2.1. MMITSS APPLICATIONS

i. **MMITSS_OBE_PriorityRequestGenerator_field**

The OBE_PriorityRequestGenerator is a critical MMITSS component that is responsible for determining a vehicle's eligibility for priority, the level of priority

to request, estimating the desired service time (e.g. estimated time of arrival at the stop bar). It is also responsible for updating any of the request information if there is a change in status or data. There are no arguments for this application.

4.3.2.2. CONFIGURATION FILES

i. vehicleid.txt

The content of the file shows the vehicle ID, vehicle type and vehicle name. The vehicle Type 1 is associated to EV, type 2 is associated to transit, and type 3 is associated to truck. Example of content of vehicleid.txt is as follows:

```
1233 2 OBE33
```

ii. signal_status.txt (Auto-Generated)

This file is auto-generated during the MMITSS operations. The content includes 8 numbers that show the status of each phase. Sample content of this file is as follows:

```
301 signal_status 1 3 1 1 1 3 1 1
```

The intersection ID in this example is 301. The first 8 digits after signal_status show the signal status. Digit 3 indicates the phase is green, digit 1 indicates it is red, and digit 4 indicates it is yellow.

iii. Intersection_maps.txt (Auto-Generated)

This file is auto-generated during the MMITSS operations. The content of the file is the list of received maps on the OBU.

iv. Intersection_MAP_<MapID>.nmap (Auto-Generated)

This file is auto-generated during the MMITSS operations. The content of the file is the map description file with MAP ID. For details of the MAP file, refer Appendix-I

v. psm.txt (Auto-Generated)

This file is auto-generated during the MMITSS operations. The content of the file indicates all of the active priority requests at the intersection. The first line shows the number of requests. The rest lines are the information of each request such as vehicle id, type, ETA, inlane, outlane, start time of service, end time of service, and vehicle status.

vi. ActiveMAP.txt (Auto-Generated)

The content of the file indicates which intersection the vehicle is approaching and which one it is leaving. Sample content of this file is as follows:

```
301 1428888550 1
302 1428888550 0
```

Each row in this file shows the intersection ID of the received MAP, the last time that MAP is received, and an indicator that shows the ID of the approaching intersection. If this indicator is 1, the vehicle is approaching the intersection; if the indicator is -1, the vehicle is leaving the intersection; if the indicator is 0, the

intersection is not the closest approaching intersection for the vehicle. In this example, the vehicle is in the range of two intersection 301 and 302. But the closest approaching intersection is intersection 301.

vii. busStopsRange.txt

This file is used only for transit vehicles. It indicates the distance between each bus stop in the route of the bus to the next intersection. The bus stop distance to intersection is important only when it is less than 300 meters. The general syntax for the busStopsRange.txt file is as follows:

```
Route_Name ##name of the transit route
Number_of_Intersections ##Number of intersections a transit vehicle crosses
while on the route.
x1 y1 ##SignalController_ID(x1) Number_IngressApproaches on Intersection(y1)
1 m ##Distance(m)between the last node on the NMAP and stop Bar_Approach(1)
.
.
.
y m ##Distance(m)between the last node on the NMAP and stop Bar_Approach(y)

x2 y2 ##SignalController_ID(x2) Number_IngressApproaches on Intersection(y2)
1 m ##Distance(m)between the last node on the NMAP and stop Bar_Approach(1)
.
.
.
y m ##Distance(m)between the last node on the NMAP and stop Bar_Approach(y)
.
.
```

The example of content of busStopRange.txt file is as follows:

```
Route_Name Redwood
Number_of_Intersections 2
7090 4
1 9999
3 9999
5 9999
7 9999
7091 4
1 9999
3 9999
5 9999
7 160
```

5. TROUBLESHOOTING

Common problems faced during the implementation of MMITSS are addressed in this sections with proposed solutions.

5.1. REDUCED RESPONSE-TIME OF SYSTEM

If it is observed that the MMITSS system is slowed down, following steps may be followed:

1. Delete the existing Log files of MMITTS. Directory of log files is located at the path:

   `nojournal/bin/log/`

   Files can be deleted using WinSCP, using following steps:

   i. Stop the MMITSS Script using following command on RSE:

   `/etc/init.d/mmitss stop`

   Or if troubleshooting on the OBE, using following command:

   `/etc/init.d/MMITSS stop`

   ii. Open WinSCP

   iii. Direct to path `nojournal/bin/log/`

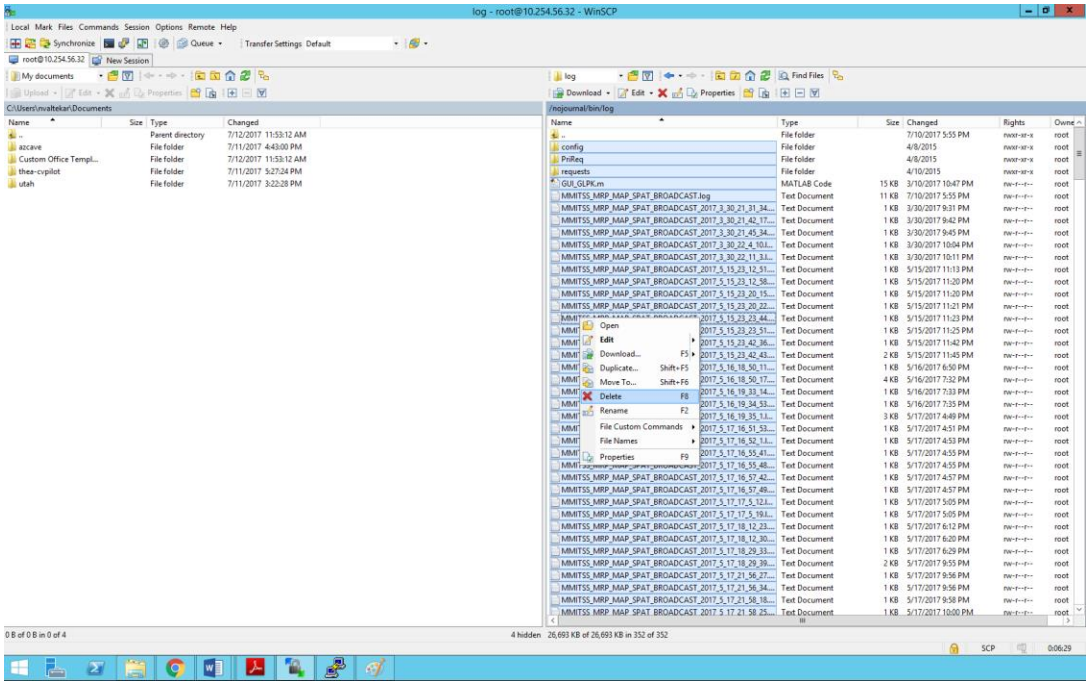


FIGURE 30: WINSCP - SELECTING ALL LOG FILES

   iv. Press `Ctrl+A` to select all files in the folder.

   v. Right-Click on any of selected files and Click on Delete button. (Figure 30)

   vi. Provide confirmation to delete files in the dialogue box. (Figure 31)

vii. Restart the *MMITSS Script*.

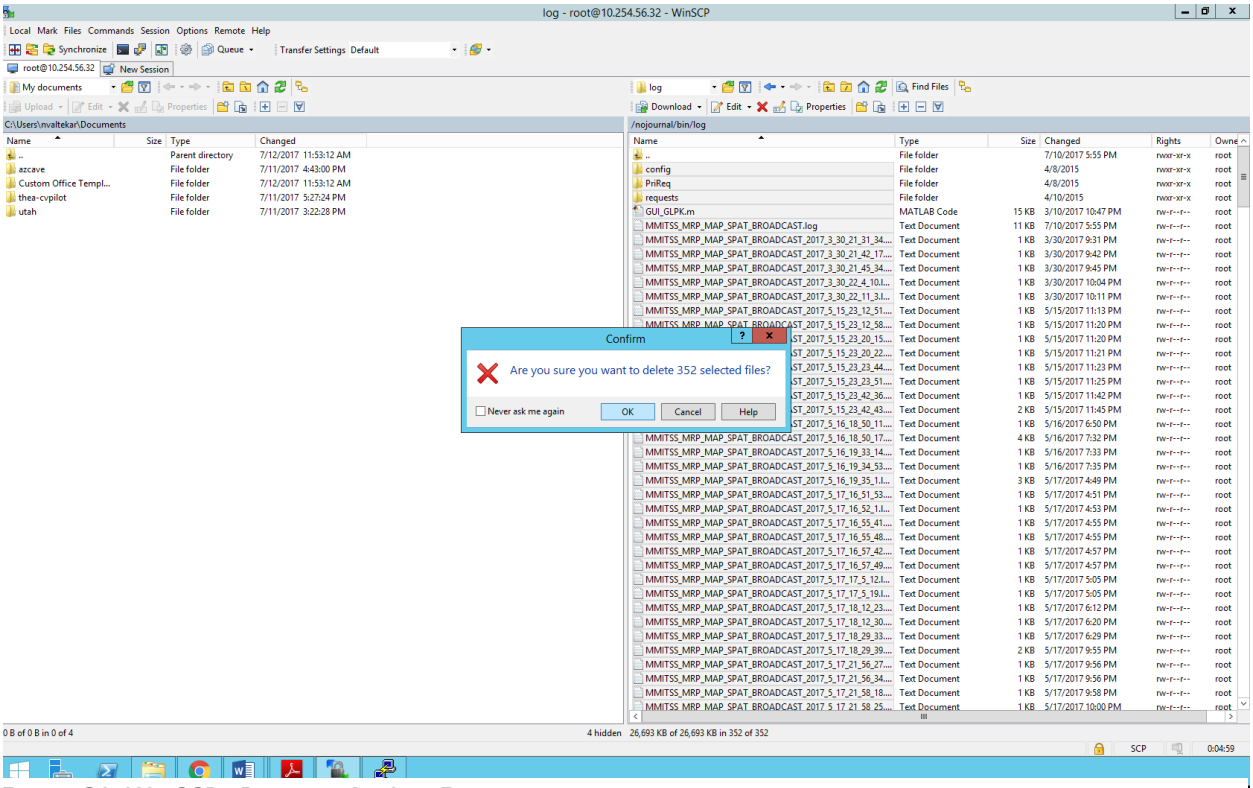


FIGURE 31: WINSCP - DELETING ALL LOG FILES

2. If the performance is still not improved, reboot the RSE/OBE and restart the MMITSS script.

5.2. MISSING MMITSS COMPONENTS

During MMITSS operations, if it is observed that some components are not working, following steps may be followed:

i. Check if all MMITSS are still running, using list of running applications, which opens in PuTTY with command `ps` (Refer Figure 32)

ii. If any application is missing from the running files, check if the *MMITSS Script* contains a start call to all required applications.

iii. If a call to any application is missing in the *MMITSS Script*, modify the *MMITSS Script* accordingly and re-start the *MMITSS Script*.

iv. If any application is found missing from the list of running files in spite of being called by the *MMITSS Script* initially, stop and re-start the MMITSS application.

v. If the applications are still missing from the list of running applications, reboot the RSU and restart the MMITSS application. Repeat the installation of MMITSS application package if necessary.

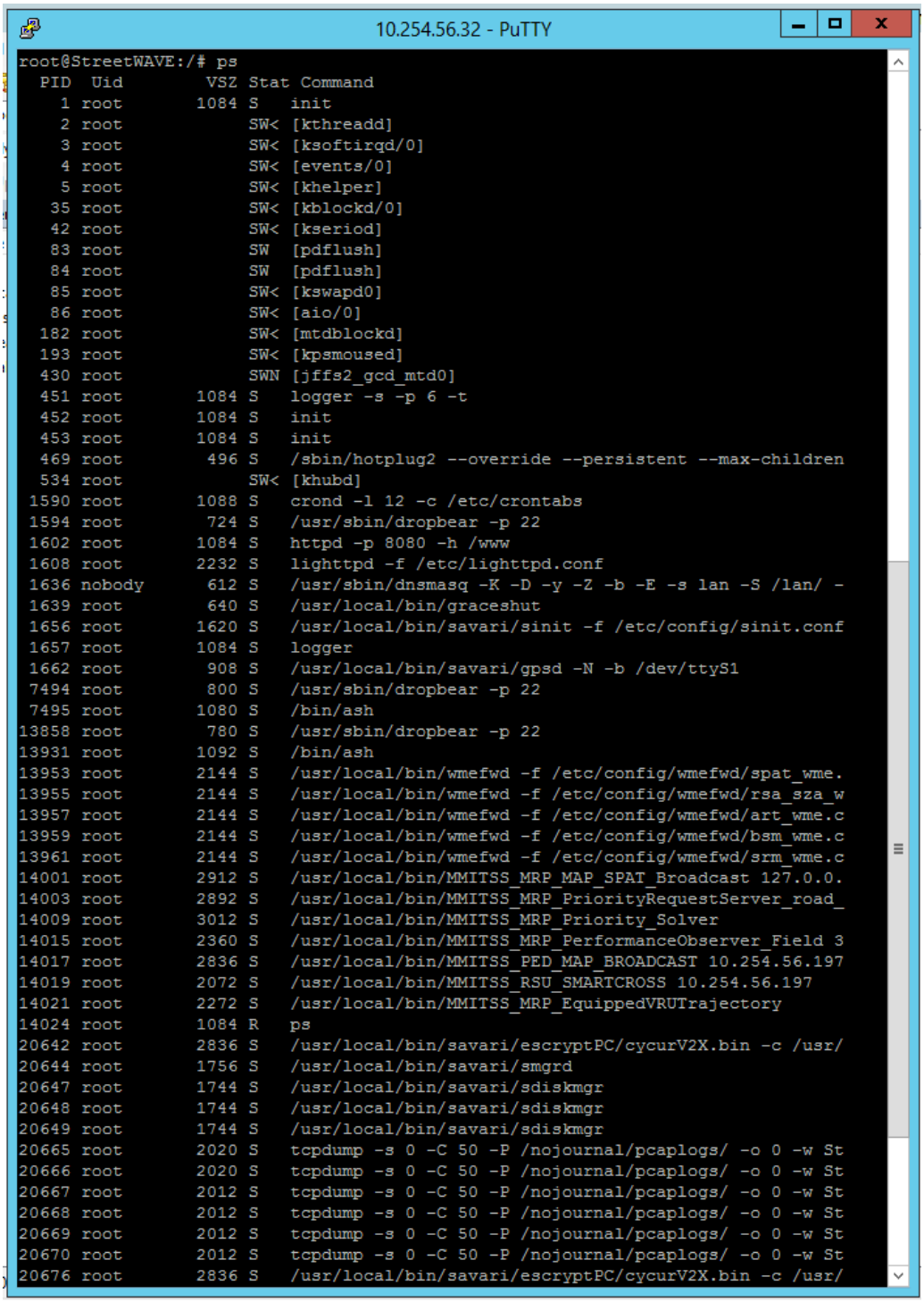


FIGURE 32: PUTTY - CHECKING ALL RUNNING APPLICATIONS

6. REFERENCES

Bai, F., Stancil, D. D., & Krishnan, H. (2010). Toward Understanding Characteristics of Dedicated Short Range Communications (DSRC) from a Perspective of Vehicular Network Engineers. In Proceedings of the Sixteenth Annual International Conference on Mobile Computing and Networking (pp. 329–340). New York, NY, USA: ACM. http://doi.org/10.1145/1859995.1860033

Battelle, " SIGNAL PHASE AND TIMING AND RELATED MESSAGE Binary Format (BLOB) Details", FHWA Office of Operations Research and Development, February 17, 2012, Rev. C.

Bullock, D., Johnson, B., Wells, R. B., Kyte, M., & Li, Z. (2004). Hardware-in-the-loop simulation. Transportation Research Part C: Emerging Technologies, 12(1), 73–89. http://doi.org/10.1016/j.trc.2002.10.002

Byrne, N., Koonce, P., Bertini, R., Pangilinan, C., & Lasky, M. (2005). Using Hardware-in-the-Loop Simulation to Evaluate Signal Control Strategies for Transit Signal Priority. Transportation Research Record: Journal of the Transportation Research Board, 1925, 227–234. http://doi.org/10.3141/1925-23

Ding, J., He, Q., Head, L., Saleem, F., & Wu, W. (2013). Development and Testing of Priority Control System in Connected Vehicle Environment. Presented at the Transportation Research Board 92nd Annual Meeting. Retrieved from https://trid.trb.org/view.aspx?id=1241792

DLR - Institute of Transportation Systems - SUMO – Simulation of Urban Mobility. (n.d.). Retrieved May 25, 2016, from http://www.dlr.de/ts/en/desktopdefault.aspx/tabid-9883/16931_read-41000/

DSRC Roadside Unit (RSU) Specifications Document v4.1 (2016) – Florida Department of Transportation. Retrieved from, http://www.fdot.gov/traffic/Doc_Library/PDF/USDOT%20RSU%20Specification%204%201_Final_R1.pdf

Electronic Code of Federal Regulations. (2017). Title 47, Chapter I, Subchapter D, Part 90, Subpart M, §90.377. Retrieved from, https://www.ecfr.gov/cgi-bin/text-idx?SID=a73a74bbd347d4f4ce80a178667ce59d&mc=true&node=se47.5.90_1377&rgn=div8

Feng, Y., Head, K.L., and Ma, W., Estimating Vehicles in the Dilemma Zone and Application to Fixed-time Coordinated Signal Optimization. Transportation Research Record, Vol. 2439, pp. 62-70, 2014.

Feng, Y., Zamanipour, M., Head, K. L., and Khoshmagham, S., “Connected Vehicle–Based Adaptive Signal Control and Applications”, Volume 2558, Transportation Research Record, Journal of the Transportation Research Board, 2016, pp. 11-19.

Gartner, N.H., "OPAC: a demand-responsive strategy for traffic signal control", Transportation Research Record: Journal of the Transportation Research Board, Vol. 906, 1983, pp. 75–81

Gettman, D., Shelby, S., Head, L., and Ghaman, R. "Overview of the ACS-Lite Adaptive Control System", 13th World Congress on Intelligent Transport Systems and Services, 2006.

Goodall, N., Smith, B., Park, B., 2013. Traffic Signal Control with Connected Vehicles. Transportation Research Record: Journal of the Transportation Research Board, Vol. 2381, pp. 65–72.

He, Q., K. L. Head, and J. Ding. Heuristic Algorithm for Priority Traffic Signal Control. Transportation Research Record: Journal of the Transportation Research Board, Vol. 2259, No. -1, Dec. 2011, pp. 1–7.

Head, L., D. Gettman, and Z. Wei. Decision Model for Priority Control of Traffic Signals. Transportation Research Record: Journal of the Transportation Research Board, Vol. 1978, Jan. 2006, pp. 169–177.

Khoshmagham, Shayan, "Real Time Performance Observation And Measurement In A Connected Vehicle Environment", July 2016.

Mirchandani, P., & Head, L. (2001). A real-time traffic signal control system: architecture, algorithms, and analysis. Transportation Research Part C: Emerging Technologies, 9(6), 415–432. http://doi.org/10.1016/S0968-090X(00)00047-4

NTCIP 1202, National Transportation Communications for ITS Protocol Object Definitions for Actuated Traffic Signal Controller (ASC) Units, v02.19. November 2005.

Pigne, Y., Danoy, G., & Bouvry, P. (2010). A platform for realistic online vehicular network management (pp. 595–599). IEEE. http://doi.org/10.1109/GLOCOMW.2010.5700389

Zamanipour, M. "A Unified Decision Framework for Multi-Modal Traffic Signal Control Optimization in a Connected Vehicle Environment", July 2016.

7. APPENDICES

7.1. CONSTRUCTION AND AN EXAMPLE OF MAP FILE

7.1.1. GENERAL DESCRIPTION OF THE PROCESS FOR DEVELOPMENT OF THE MAP DATA

The MAP data is provided in a MAP Description file (XXX.nmap) which is a text file that includes the GPS waypoints that describe the roadway geometry. A sample data file is provided in the Section 7.1.3. The GPS waypoints are collected from Google Earth (which are accurate enough for traffic control purposes, but not for safety applications). For safety applications, these waypoints could be collected using field survey equipment.

7.1.2. CONVENTIONS USED IN MAP DEPLOYMENT

1. The MAP messages developed are intended for use by MMITSS and not intended for supporting safety applications. Therefore, the approach of using Google Earth provides sufficient accuracy for traffic control applications. It is acknowledged that the MAP data must be replaced with accurate data when the system is intended to support safety applications. The Google Earth approach is used for only for expediency.
2. With DSRC, MAP messages can cover the distance of minimum of 300 meters.
3. Map and DSRC range is critical to the performance of signal priority applications (especially emergency vehicle priority). Distance, and vehicle speed, determine the time headway of the signal request message. Greater time allows the priority application to be more effective given signal control constraints (e.g. minimum green time, yellow change, red clearance, and pedestrian clearance) that can severely limit the ability of the application to give priority.
4. Using the 2009 version of the J2735, the size of the MAP Message which is directly based on the number of waypoints used to describe lanes and approaches is severely limited. This impacts the map development in two ways:
    a. In case of curved roadways, the range (distance) may need to be limited so that lane waypoints could be placed close together to capture the roadway curvature. If points were too far apart, the vehicle would compute its position on the map incorrectly and determine that it was no longer “on the map” even though it was centered in the roadway. The vehicle application (Priority Request Generator) uses a geo fencing approach when locating the vehicle on the roadway to limit the application to only send requests for priority when the vehicle was in the roadway and approaching the intersection and not in a parking lot approaching the intersection.
    b. The pedestrian application uses Wi-Fi for communications and doesn’t require map data for every traffic lane, especially at the same range (distance). A separate MAP may be defined for the pedestrian application that includes the sidewalks and crosswalks. (Sidewalks are used in the pedestrian app to help improve the positioning accuracy as pedestrians are approaching the intersection). Since the pedestrian app doesn’t need the full map data, the sidewalks and crosswalks are not included in the full MAP message. This reduces the number of waypoints included in the full MAP message.

5. In the NMAP file, it is required to define Intersection and Lane attributes which are 8-bit and 16-bit data objects which describe general attributes of intersection and lanes. Table 5 and Table 6 provide the guidelines for the bits of the MAP message attribute objects.

TABLE 5: NMAP FILE - INTERSECTION ATTRIBUTES

Bit ID	Description	0	1
0	Elevation data is included	No	Yes
1	Lane width data is included in the node list	No	Yes
2	Node data is packed in 12-bit format instead of standard 16 bit format	No	Yes
3	Node offset resolution	centimeter	decimeter
4	Intersection geometric data is included	No	Yes
5	Navigational movement data is included	No	Yes
6	Reserved	No	Yes
7	Reserved	No	Yes

TABLE 6: NMAP FILE - LANE ATTRIBUTES

Bit ID	**Description**	**0**	**1**
0	Lane is egress path	No	Yes
1	Straight maneuver permitted	No	Yes
2	Left turn maneuver permitted	No	Yes
3	Right turn maneuver permitted	No	Yes
4	Yield	No	Yes
5	No U-Turn permitted	No	Yes
6	No turn on red permitted	No	Yes
7	No stopping permitted	No	Yes

8	HOV lane	No	Yes
9	Bus only lane	No	Yes
10	Bus and Taxi only lane	No	Yes
11	Shared two-way left turn lane	No	Yes
12	Bike lane	No	Yes
13	Reserved	No	Yes
14	Reserved	No	Yes
15	Reserved	No	Yes

7.1.3. MAP DESCRIPTION FILE OF DAISY MOUNTAIN DR AND GAVILAN PEAK

An example MAP of Daisy Mountain Dr and Gavilan Peak at Anthem is shown in Figure 33 and Figure 34.

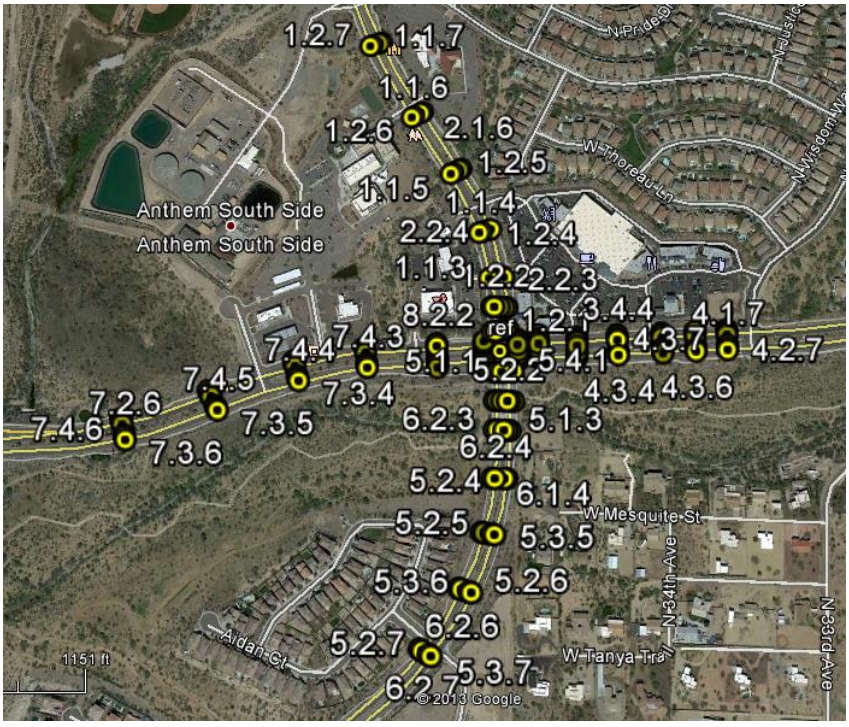


FIGURE 33: AREA MAP - DAISY MOUNTAIN DR AND GAVILANB PEAK

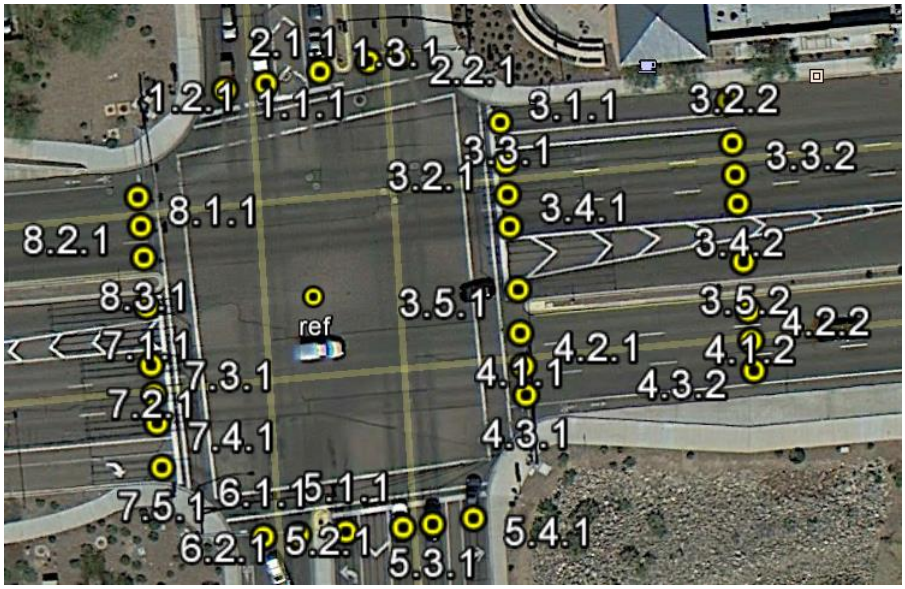


FIGURE 34: INTERSECTION MAP - DAISY MOUNTAIN DR AND GAVILAN PEAK

MAP description file example: Daisy Mountain Dr and Gavilan Peak

```
MAP_Name    Daisy_Gav.nmap
RSU_ID      Daisy_Gav
IntersectionID    1
Intersection_attributes 00110011     /*elevation: Yes, lane width: Yes, Node
date 16 bits, node offset solution: cm, geometry: Yes, navigation: Yes*/
Reference_point   33.8429408      -112.1352055   5350          /*lat,   long,
evelation(decimeter)*/
No_Approach 8
Approach    1
Approach_type     1                    /*1: approach, 2: engress*/
No_lane     3
Lane  1.1
Lane_ID     1
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000000101010     /*not egress path, straight and right
turn permitted, turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    8
1.1.1 33.8431696  -112.1353224
1.1.2 33.8434774  -112.1353437
1.1.3 33.8438323  -112.1353991
1.1.4 33.8444039  -112.1355749
1.1.5 33.8451492  -112.1360386
1.1.6 33.8458580  -112.1366679
1.1.7 33.8467824  -112.1373606
1.1.8 33.8477167  -112.1378525
No_Conn_lane      2
6.1   4     /*Lane 3.1, Straight head */
8.1   3     /*Lane 4.1, Right Turn */
end_lane
```

```
Lane  1.2
Lane_ID     2
Lane_type   1
Lane_attributes   0000000001100010    /*not egress path, straight permitted, no
u-turn */
Lane_width  365
No_nodes    8
1.2.1 33.8431764  -112.1352686
1.2.2 33.8434765  -112.1353073
1.2.3 33.8438387  -112.1353569
1.2.4 33.8444167  -112.1355321
1.2.5 33.8451668  -112.1360031
1.2.6 33.8458773  -112.1366308
1.2.7 33.8467969  -112.1373230
1.2.8 33.8477313  -112.1378162
No_Conn_lane      1
6.2   4     /*Lane 3.2, Straight head */
end_lane
Lane  1.3
Lane_ID     3
Lane_type   1
Lane_attributes   0000000001100100    /*not egress path, left turn permitted,
no u-turn */
Lane_width  365
No_nodes    3
1.3.1 33.8431898  -112.1351964
1.3.2 33.8434776  -112.1352271
1.3.3 33.8438435  -112.1353141
No_Conn_lane      1
4.1   2     /*Lane 4.1, Left turn */
end_lane
end_approach
Approach    2
Approach_type     2                    /*1: approach, 2: engress*/
No_lane     2
Lane  2.1
Lane_ID     4
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet*/
No_nodes    8
2.1.1 33.8432011  -112.1351319
2.1.2 33.8434776  -112.1351605
2.1.3 33.8438497  -112.1352225
2.1.4 33.8444425  -112.1354401
2.1.5 33.8452071  -112.1359128
2.1.6 33.8459196  -112.1365366
2.1.7 33.8468263  -112.1372213
2.1.8 33.8477607  -112.1377204
No_Conn_lane      0     /* no connected lane because it is a engress lane*/
end_lane
Lane  2.2
Lane_ID     5
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
```

```
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    8
2.2.1 33.8432066  -112.1350907
2.2.2 33.8434768  -112.1351143
2.2.3 33.8438515  -112.1351818
2.2.4 33.8444532  -112.1354009
2.2.5 33.8452233  -112.1358807
2.2.6 33.8459371  -112.1365005
2.2.7 33.8468394  -112.1371844
2.2.8 33.8477741  -112.1376817
No_Conn_lane      0     /* no connected lane because it is a engress lane*/
end_lane
end_approach
Approach    3
Approach_type     1                    /*1: approach, 2: engress*/
No_lane     5
Lane  3.1
Lane_ID     6
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000000111000    /*not engress path, right-turn permitted,
yield, turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    2
3.1.1 33.8431382  -112.1349523
3.1.2 33.8431582  -112.1346514
No_Conn_lane      1
2.2   3     /*Lane 2.2, Right turn */
end_lane
Lane  3.2
Lane_ID     7
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not engress path, straight permitted,
no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
3.2.1 33.8430894  -112.1349488
3.2.2 33.8431114  -112.1346463
3.2.3 33.8431517  -112.1340837
3.2.4 33.8431997  -112.1334985
3.2.5 33.8432431  -112.1328550
3.2.6 33.8432802  -112.1323663
3.2.7 33.8433159  -112.1319029
No_Conn_lane      1
8.1   4     /*Lane 8.1, Straight */
end_lane
Lane  3.3
Lane_ID     8
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not engress path, straight permitted,
no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
```

```
3.3.1 33.8430540  -112.1349466
3.3.2 33.8430763  -112.1346428
3.3.3 33.8431171  -112.1340816
3.3.4 33.8431633  -112.1334942
3.3.5 33.8432142  -112.1328517
3.3.6 33.8432489  -112.1323636
3.3.7 33.8432820  -112.1319002
No_Conn_lane      1
8.2   4     /*Lane 8.2, Straight */
end_lane
Lane  3.4
Lane_ID     9
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not engress path, straight permitted,
no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
3.4.1 33.8430187  -112.1349437
3.4.2 33.8430442  -112.1346394
3.4.3 33.8430848  -112.1340795
3.4.4 33.8431309  -112.1334922
3.4.5 33.8431820  -112.1328476
3.4.6 33.8432166  -112.1323595
3.4.7 33.8432498  -112.1318975
No_Conn_lane      1
8.3   4     /*Lane 8.3, Straight */
end_lane
Lane  3.5
Lane_ID     10
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001110100    /*not engress path, left permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    3
3.5.1 33.8429492  -112.1349325
3.5.2 33.8429799  -112.1346320
3.5.3 33.8430372  -112.1340776
No_Conn_lane      1
6.2 2    /*Lane 6.2, Left turn */
end_lane
end_approach
Approach    4
Approach_type     2                   /*1: approach, 2: engress*/
No_lane     3
Lane  4.1
Lane_ID     11
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
4.1.1 33.8429010  -112.1349296
4.1.2 33.8429257  -112.1346255
4.1.3 33.8429644  -112.1340767
```

```
4.1.4 33.8430094  -112.1334783
4.1.5 33.8430560  -112.1328307
4.1.6 33.8430959  -112.1323493
4.1.7 33.8431286  -112.1318916
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
Lane  4.2
Lane_ID     12
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
4.2.1 33.8428642  -112.1349256
4.2.2 33.8428925  -112.1346211
4.2.3 33.8429315  -112.1340754
4.2.4 33.8429781  -112.1334732
4.2.5 33.8430235  -112.1328263
4.2.6 33.8430629  -112.1323462
4.2.7 33.8430950  -112.1318885
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
Lane  4.3
Lane_ID     13
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
4.3.1 33.8428323  -112.1349218
4.3.2 33.8428591  -112.1346168
4.3.3 33.8429002  -112.1340742
4.3.4 33.8429459  -112.1334691
4.3.5 33.8429909  -112.1328221
4.3.6 33.8430303  -112.1323432
4.3.7 33.8430634  -112.1318859
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
end_approach
Approach    5
Approach_type     1                   /*1: approach, 2: engress*/
No_lane     4
Lane  5.1
Lane_ID     14
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100100     /*not egress path, left permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    3
5.1.1 33.8426804  -112.1351622
5.1.2 33.8423323  -112.1351466
5.1.3 33.8419788  -112.1350981
No_Conn_lane      1
8.3 2    /*Lane 8.3, Left turn */
```

```
end_lane
Lane  5.2
Lane_ID     15
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010     /*not egress path, left permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    9
5.2.1 33.8426850  -112.1350861
5.2.2 33.8423323  -112.1350643
5.2.3 33.8419797  -112.1350482
5.2.4 33.8414066  -112.1350652
5.2.5 33.8407512  -112.1352137
5.2.6 33.8400770  -112.1355026
5.2.7 33.8393485  -112.1360278
5.2.8 33.8387132  -112.1367453
5.2.9 33.8380366  -112.1377007
No_Conn_lane      1
2.1 4    /*Lane 2.1, Straight */
end_lane
Lane  5.3
Lane_ID     16
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    9
5.3.1 33.8426886  -112.1350463
5.3.2 33.8423317  -112.1350253
5.3.3 33.8419801  -112.1350044
5.3.4 33.8414081  -112.1350264
5.3.5 33.8407461  -112.1351733
5.3.6 33.8400680  -112.1354663
5.3.7 33.8393328  -112.1359922
5.3.8 33.8386928  -112.1367145
5.3.9 33.8380175  -112.1376693
No_Conn_lane      1
2.2 4    /*Lane 2.2, Straight */
end_lane
Lane  5.4
Lane_ID     17
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000000111000     /*not egress path, right permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    2
5.4.1 33.8426951  -112.1349918
5.4.2 33.8423306  -112.1349676
No_Conn_lane      1
4.3 3    /*Lane 4.3, right turn */
end_lane
end_approach
Approach    6
Approach_type     2                    /*1: approach, 2: engress*/
```

```
No_lane     2
Lane  6.1
Lane_ID     18
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    9
6.1.1 33.8426741  -112.1352729
6.1.2 33.8423321  -112.1352513
6.1.3 33.8419801  -112.1352359
6.1.4 33.8414022  -112.1352401
6.1.5 33.8407680  -112.1353608
6.1.6 33.8401121  -112.1356476
6.1.7 33.8394079  -112.1361541
6.1.8 33.8387952  -112.1368640
6.1.9 33.8381254  -112.1378031
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
Lane  6.2
Lane_ID     19
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    9
6.2.1 33.8426767  -112.1352262
6.2.2 33.8423322  -112.1352121
6.2.3 33.8419800  -112.1351883
6.2.4 33.8414018  -112.1351989
6.2.5 33.8407625  -112.1353226
6.2.6 33.8401018  -112.1356098
6.2.7 33.8393883  -112.1361182
6.2.8 33.8387725  -112.1368310
6.2.9 33.8381035  -112.1377706
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
end_approach
Approach    7
Approach_type     1                   /*1: approach, 2: engress*/
No_lane     5
Lane  7.1
Lane_ID     20
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001110100    /*not engress path, left-turn permitted,
yield, no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    2
7.1.1 33.8429302  -112.1354272
7.1.2 33.8428636  -112.1361080
No_Conn_lane      1
2.1   2     /*Lane 2.1, Left turn */
end_lane
Lane  7.2
```

```
Lane_ID     21
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not engress path, straight permitted,
no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
7.2.1 33.8428654  -112.1354214
7.2.2 33.8428135  -112.1361033
7.2.3 33.8427284  -112.1371296
7.2.4 33.8425319  -112.1381276
7.2.5 33.8421312  -112.1392849
7.2.6 33.8417334  -112.1405642
7.2.7 33.8415676  -112.1416605
No_Conn_lane      1
4.1   4     /*Lane 4.1, Straight */
end_lane
Lane  7.3
Lane_ID     22
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not engress path, straight permitted,
no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
7.3.1 33.8428326  -112.1354178
7.3.2 33.8427817  -112.1361002
7.3.3 33.8426974  -112.1371231
7.3.4 33.8425013  -112.1381152
7.3.5 33.8421005  -112.1392649
7.3.6 33.8417018  -112.1405498
7.3.7 33.8415360  -112.1416550
No_Conn_lane      1
4.2   4     /*Lane 4.2, Straight */
end_lane
Lane  7.4
Lane_ID     23
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100010    /*not engress path, straight permitted,
no turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
7.4.1 33.8428010  -112.1354142
7.4.2 33.8427477  -112.1360977
7.4.3 33.8426653  -112.1371154
7.4.4 33.8424691  -112.1381003
7.4.5 33.8420734  -112.1392453
7.4.6 33.8416709  -112.1405380
7.4.7 33.8415027  -112.1416507
No_Conn_lane      1
4.3   4     /*Lane 4.3, Straight */
end_lane
Lane  7.5
Lane_ID     24
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
```

```
Lane_attributes   0000000001111000    /*not engress path, right turn permitted,
yeild, turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    2
7.5.1 33.8427514  -112.1354085
7.5.2 33.8427004  -112.1360939
No_Conn_lane      1
6.1   3     /*Lane 6.1, Straight */
end_lane
end_approach
Approach    8
Approach_type     2                    /*1: approach, 2: engress*/
No_lane     3
Lane  8.1
Lane_ID     25
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
8.1.1 33.8430543  -112.1354386
8.1.2 33.8429994  -112.1361246
8.1.3 33.8429109  -112.1371694
8.1.4 33.8427083  -112.1381982
8.1.5 33.8423043  -112.1394037
8.1.6 33.8419244  -112.1406316
8.1.7 33.8417790  -112.1416925
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
Lane  8.2
Lane_ID     26
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
8.2.1 33.8430190  -112.1354348
8.2.2 33.8429646  -112.1361202
8.2.3 33.8428790  -112.1371606
8.2.4 33.8426796  -112.1381817
8.2.5 33.8422668  -112.1393744
8.2.6 33.8418905  -112.1406170
8.2.7 33.8417423  -112.1416862
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
Lane  8.3
Lane_ID     27
Lane_type   1        /*1 to 5, for this intersection all 1: motorized vehicle
lane */
Lane_attributes   0000000001100011     /*egress path, straight permitted, no
turn on red, no u-turn */
Lane_width  365      /* in centimeter =12 feet
No_nodes    7
8.3.1 33.8429844  -112.1354310
8.3.2 33.8429334  -112.1361169
```

```
8.3.3 33.8428492  -112.1371516
8.3.4 33.8426475  -112.1381667
8.3.5 33.8422380  -112.1393535
8.3.6 33.8418552  -112.1406033
8.3.7 33.8417100  -112.1416820
No_Conn_lane      0    /* no connected lane because it is a engress lane*/
end_lane
end_approach
end_map
```

8. RECORD OF CHANGES

Version Number	Date	Figure, Table, or Paragraph	Brief Description	Change Request Number
1.0	06/07/2017	N/A	Initial Draft	0
2.0	07/20/2017	All	1. Added instructions to install setup the signal controller for MMITSS implementation. 2. Added general instructions to communicate with MMITSS RSE and OBE. 3. Changed the format of instructions. Added visual assists to support the instructions. 4. Updated the lists of required configuration files. Separated auto-generated files from the files required to be built manually. 5. Added a troubleshooting guide for commonly faced problems during MMITSS operations. 6. Added an Appendix-I showing construction of MAP file.	1